CMCMIS_SIMPLIFIED

PHASE 11 - PDF GENERATION

100+ Page End-to-End Sealed Technical Documentation

Job Card Certificate + Job Card Details + Job Request Details

DS LOCKED SEAL

This document records Phase 11 as a sealed production-grade implementation phase. It explains the logic development, database reasoning, backend design, frontend wiring, PDFKit rendering strategy, RBAC gates, smoke verification, and future handover controls. The documentation is written in the DS senior-engineering style: step-by-step, insights-based, diagram-heavy, table-backed, definition-driven, and implementation-aware.

Seal Field	Locked Value
Phase	11 - PDF Generation
Backend objective	Stream three real-time PDFs from sealed CMCMIS legacy/MVP data
PDF #1	Job Card Certificate - locked single-page TIMCD layout
PDF #2	Job Card Full Details - own-design multi-page internal archive
PDF #3	Job Request Details - own-design multi-page request lifecycle archive
Permission migration	410__phase11_pdf_permissions.sql
Smoke verification	23/23 green
Zero-write proof	audit_log unchanged + JM_UPDATED_ON unchanged
Frontend result	Download Report live + Full Details + Request PDF buttons

Prepared as a technical-documentation artifact for CMCMIS_SIMPLIFIED / TIMCD / ISRO-style maintenance and calibration workflows.

Document Control and Sealed Scope

Purpose

The purpose of this Phase 11 document is to preserve not only what was coded, but why each design choice was taken. Phase 11 is the PDF generation layer over sealed operational data. It does not redesign job cards, job requests, or the database. It reads the sealed state and emits controlled documents.

Control Item	Description
Status	SEALED / LOCKED
Source transcript	Uploaded Pasted text implementation transcript
Main module	Backend src/modules/pdf
Frontend touchpoints	Job Card Detail Header + Job Request Detail Header
Database touchpoint	Additive permission seed only, no schema mutation
Verification style	Runtime HTTP smoke, RBAC, PDF magic bytes, page count, zero-write checks

Reading Rule

Read the document like a senior engineer handover: first understand the business need, then the sealed constraints, then the data model, then the API endpoints, then the renderers, then the frontend triggers, then the verification matrix.

Manual Table of Contents

Executive Summary - What Phase 11 Achieved

DS Insight

Phase 11 converted the sealed CMCMIS operational state into downloadable PDF artifacts. The earlier phases already created the job request lifecycle, job card lifecycle, observations, task checklists, documents, maintenance rows, spares, reports, and analytics. Phase 11 sits above that sealed base and generates official documents without mutating database state. This is important because a PDF download must never become a hidden workflow transition.

Phase 11 Position in the Sealed Project Stack

```
+----------------------------+
| Phase 10 Reports & Analytics |
+----------------------------+
              |
              v
+----------------------------+
| Phase 11 PDF Generation      |  <-- this document
+----------------------------+
              | reads only
              v
+----------------------------+
| Phase 9 Job Card Detail      |
+----------------------------+
              | linked from
              v
+----------------------------+
| Phase 7 JR Approval + JC     |
+----------------------------+
```

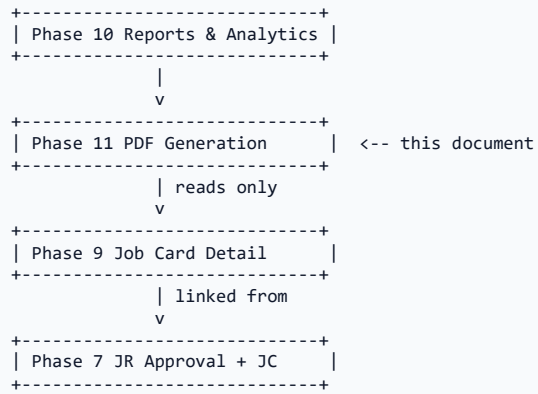
Question	Locked Answer
Does Phase 11 alter business state?	No
Does Phase 11 persist generated PDFs?	No
Does Phase 11 expose stale data?	No - each request reads live DB
Does Phase 11 require new core schema?	No - only additive permissions

Executive Summary - Three Document Types

DS Insight

The implementation produced three different PDFs. The first is the Job Card Certificate, a locked single-page document faithful to the TIMCD job request/calibration form. The second is the full Job Card Details report, a multi-page archive covering all job card tabs and Phase 9 child tables. The third is the Job Request Details report, a multi-page request archive showing classification, submitter, equipment snapshot, accessories, approval/rejection data, linked job card, and status history.

Phase 11 Position in the Sealed Project Stack



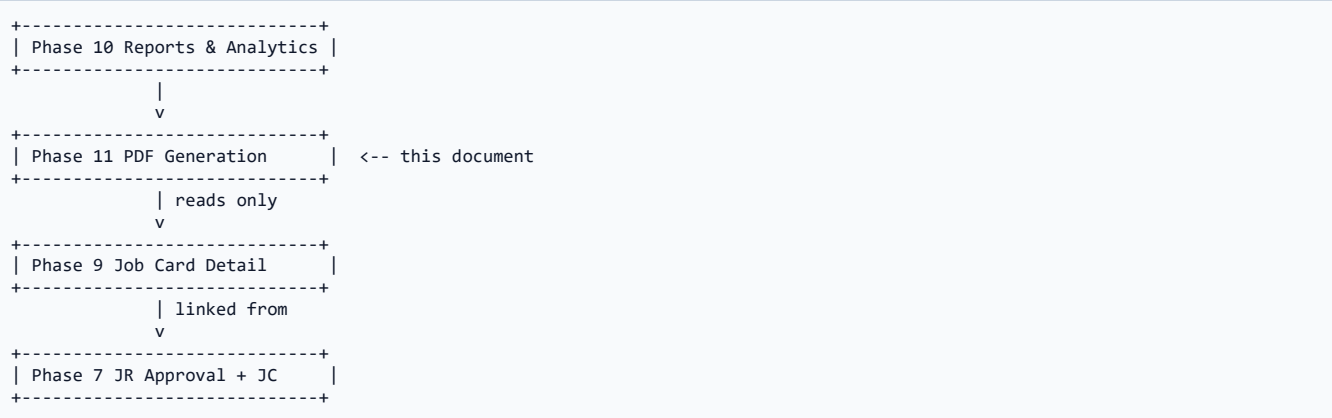
Question	Locked Answer
Does Phase 11 alter business state?	No
Does Phase 11 persist generated PDFs?	No
Does Phase 11 expose stale data?	No - each request reads live DB
Does Phase 11 require new core schema?	No - only additive permissions

Executive Summary - Senior Engineering Principle

DS Insight

The central principle was read-only generation. The PDF layer reads the database, performs permission checks and eligibility checks, streams a response, and exits. It does not insert audit rows, create files on disk, update updated_at columns, or persist generated PDFs. The smoke test explicitly proved this by checking audit_log and cmms_jobcard_mst timestamps before and after all requests.

Phase 11 Position in the Sealed Project Stack



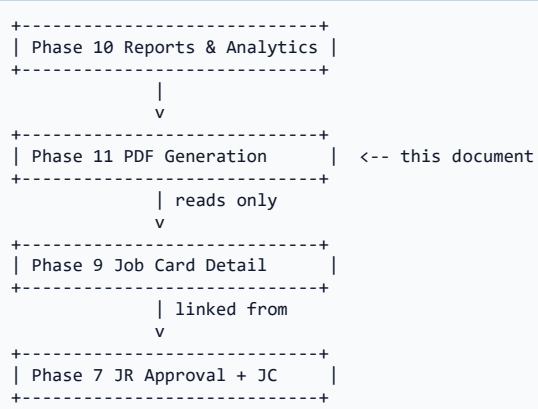
Question	Locked Answer
Does Phase 11 alter business state?	No
Does Phase 11 persist generated PDFs?	No
Does Phase 11 expose stale data?	No - each request reads live DB
Does Phase 11 require new core schema?	No - only additive permissions

Executive Summary - Why This Matters

DS Insight

In a government-grade calibration system, a downloadable PDF becomes a formal artifact. If the PDF layer were allowed to write state, it could accidentally become a hidden approval mechanism or a hidden audit mechanism. Phase 11 keeps document generation separate from workflow progression. That separation makes the system safer, easier to reason about, and easier to certify.

Phase 11 Position in the Sealed Project Stack



Question	Locked Answer
Does Phase 11 alter business state?	No
Does Phase 11 persist generated PDFs?	No
Does Phase 11 expose stale data?	No - each request reads live DB
Does Phase 11 require new core schema?	No - only additive permissions

Definition 1: PDFKit Streaming

Definition

A server-side PDF generation approach where the document is rendered directly into the HTTP response stream. This avoids saving temporary files and supports fresh data on every request.

Why it exists in Phase 11

PDFKit Streaming is not an isolated term; it controls how the PDF generation module remains safe, predictable, and compatible with the sealed CMCMIS database and workflow stack.

Dimension	Explanation
Business meaning	PDFKit Streaming helps convert operational records into trustworthy documents.
Backend meaning	PDFKit Streaming guides module boundaries and error handling.
Database meaning	PDFKit Streaming avoids uncontrolled writes or schema drift.
Testing meaning	PDFKit Streaming is validated through smoke checks and/or code structure.

Definition 2: Certificate Eligibility

Definition

The rule that the locked Job Card Certificate can be downloaded only when the job card status is COMPLETED or VERIFIED_CLOSED. Any other status returns 409 CONFLICT.

Why it exists in Phase 11

Certificate Eligibility is not an isolated term; it controls how the PDF generation module remains safe, predictable, and compatible with the sealed CMCMIS database and workflow stack.

Dimension	Explanation
Business meaning	Certificate Eligibility helps convert operational records into trustworthy documents.
Backend meaning	Certificate Eligibility guides module boundaries and error handling.
Database meaning	Certificate Eligibility avoids uncontrolled writes or schema drift.
Testing meaning	Certificate Eligibility is validated through smoke checks and/or code structure.

Definition 3: Repo Alias Discipline

Definition

A codebase boundary rule: real legacy column names like JM_* and JR_* appear only inside repository SQL. Service, controller, and templates consume canonical snake_case names.

Why it exists in Phase 11

Repo Alias Discipline is not an isolated term; it controls how the PDF generation module remains safe, predictable, and compatible with the sealed CMCMIS database and workflow stack.

Dimension	Explanation
Business meaning	Repo Alias Discipline helps convert operational records into trustworthy documents.
Backend meaning	Repo Alias Discipline guides module boundaries and error handling.
Database meaning	Repo Alias Discipline avoids uncontrolled writes or schema drift.
Testing meaning	Repo Alias Discipline is validated through smoke checks and/or code structure.

Definition 4: Zero-Write Proof

Definition

A verification pattern that compares audit_log and job-card timestamp values before and after PDF requests to prove the feature did not mutate business state.

Why it exists in Phase 11

Zero-Write Proof is not an isolated term; it controls how the PDF generation module remains safe, predictable, and compatible with the sealed CMCMIS database and workflow stack.

Dimension	Explanation
Business meaning	Zero-Write Proof helps convert operational records into trustworthy documents.
Backend meaning	Zero-Write Proof guides module boundaries and error handling.
Database meaning	Zero-Write Proof avoids uncontrolled writes or schema drift.
Testing meaning	Zero-Write Proof is validated through smoke checks and/or code structure.

Definition 5: Row-Level Scope

Definition

The visibility model where a Normal User can download only their own Job Request details. Foreign IDs collapse to 404 to avoid leaking existence.

Why it exists in Phase 11

Row-Level Scope is not an isolated term; it controls how the PDF generation module remains safe, predictable, and compatible with the sealed CMCMIS database and workflow stack.

Dimension	Explanation
Business meaning	Row-Level Scope helps convert operational records into trustworthy documents.
Backend meaning	Row-Level Scope guides module boundaries and error handling.
Database meaning	Row-Level Scope avoids uncontrolled writes or schema drift.
Testing meaning	Row-Level Scope is validated through smoke checks and/or code structure.

Definition 6: Typographic Logo Fallback

Definition

A safe fallback for ISRO/SAC visual identity: if official PNG/JPG logo files are absent, the PDF renders framed text seals instead of embedding unlicensed binary assets.

Why it exists in Phase 11

Typographic Logo Fallback is not an isolated term; it controls how the PDF generation module remains safe, predictable, and compatible with the sealed CMCMIS database and workflow stack.

Dimension	Explanation
Business meaning	Typographic Logo Fallback helps convert operational records into trustworthy documents.
Backend meaning	Typographic Logo Fallback guides module boundaries and error handling.
Database meaning	Typographic Logo Fallback avoids uncontrolled writes or schema drift.
Testing meaning	Typographic Logo Fallback is validated through smoke checks and/or code structure.

Definition 7: Two-Phase Controller Flow

Definition

A pattern where service.prepare loads and validates before any response bytes are written; only after success does the controller set headers and start the PDF stream.

Why it exists in Phase 11

Two-Phase Controller Flow is not an isolated term; it controls how the PDF generation module remains safe, predictable, and compatible with the sealed CMCMIS database and workflow stack.

Dimension	Explanation
Business meaning	Two-Phase Controller Flow helps convert operational records into trustworthy documents.
Backend meaning	Two-Phase Controller Flow guides module boundaries and error handling.
Database meaning	Two-Phase Controller Flow avoids uncontrolled writes or schema drift.
Testing meaning	Two-Phase Controller Flow is validated through smoke checks and/or code structure.

Definition 8: DSRC Block

Definition

Decision, Structure, Reason, Consequence. This documentation uses DSRC blocks to record why each technical call is locked.

Why it exists in Phase 11

DSRC Block is not an isolated term; it controls how the PDF generation module remains safe, predictable, and compatible with the sealed CMCMIS database and workflow stack.

Dimension	Explanation
Business meaning	DSRC Block helps convert operational records into trustworthy documents.
Backend meaning	DSRC Block guides module boundaries and error handling.
Database meaning	DSRC Block avoids uncontrolled writes or schema drift.
Testing meaning	DSRC Block is validated through smoke checks and/or code structure.

Step 0 - Phase 9 Schema Introspection

Description

Before writing the PDF module, the implementation introspected the Phase 9 job card columns and child tables. This avoided guessing table names or column names. The introspection identified 96 columns on cmms_jobcard_mst plus child tables for maintenance, spares, checklist, documents, observations, and history.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 0 - Phase 9 Schema Introspection.

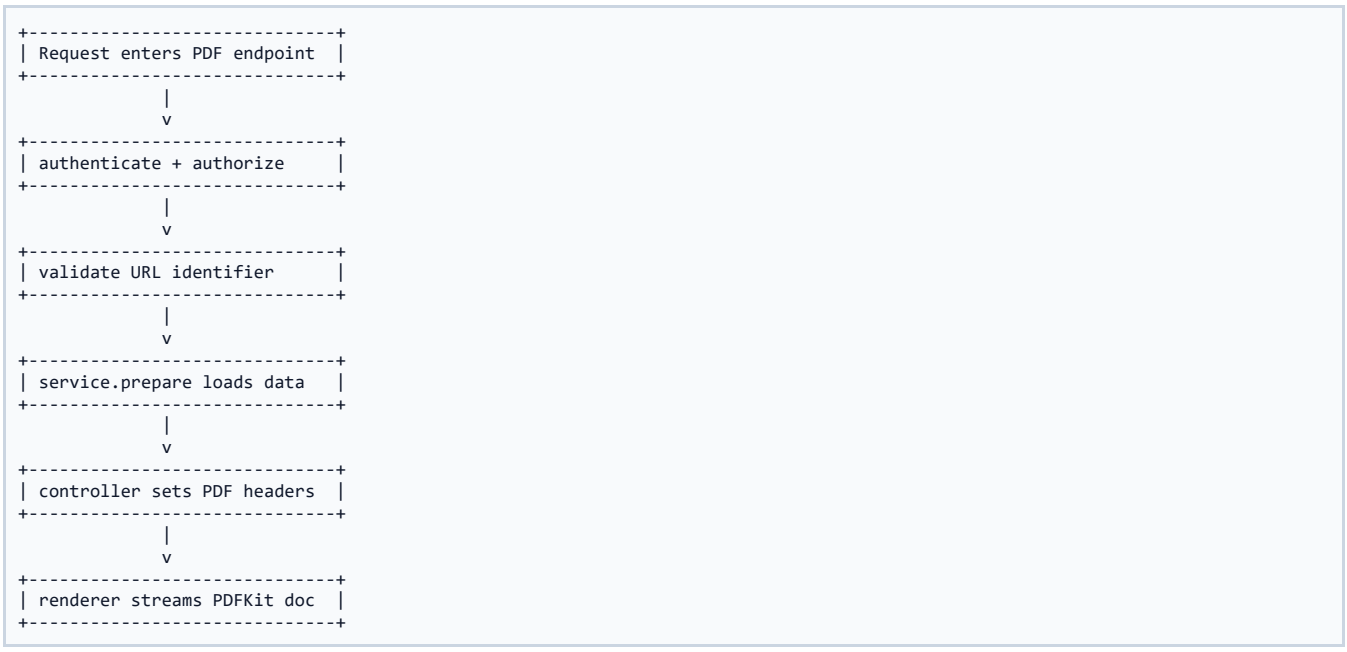
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.

Reason: preserve sealed data rules, role-aware access, and predictable document generation.

Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram



Step 1 - Existing UI Button Discovery

Description

The frontend already had a disabled Download Report button on the Job Card Detail header. Phase 11 used that exact user-facing affordance instead of adding a new confusing entry point. This preserved UI continuity and made Phase 11 feel like the planned completion of a previously reserved feature.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 1 - Existing UI Button Discovery.

Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.

Reason: preserve sealed data rules, role-aware access, and predictable document generation.

Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Step 2 - Permission Design

Description

The previous generic permission job_card:generate-pdf existed as a legacy placeholder. Phase 11 introduced granular permissions: job_card:download-certificate, job_card:download-details, and job_request:download-details. This allowed different roles to access different PDF types.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 2 - Permission Design.

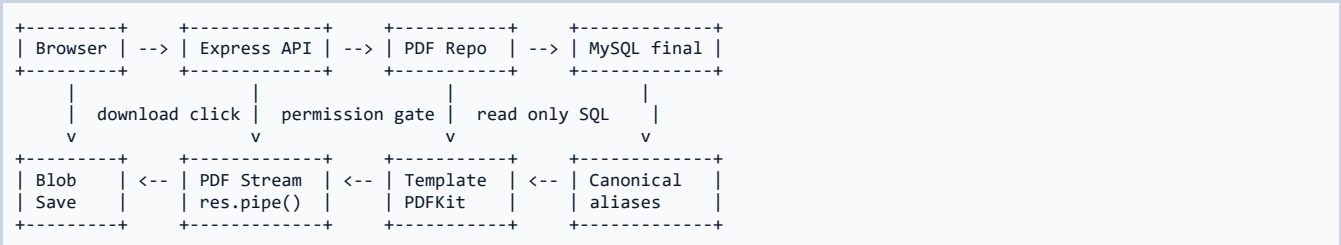
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.

Reason: preserve sealed data rules, role-aware access, and predictable document generation.

Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram



Step 3 - Additive Migration 410

Description

The migration used INSERT IGNORE so repeated runs remain idempotent. It added only permissions and role_permission grants. It did not alter job request or job card tables. This followed the sealed-schema doctrine and preserved legacy production data.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 3 - Additive Migration 410.

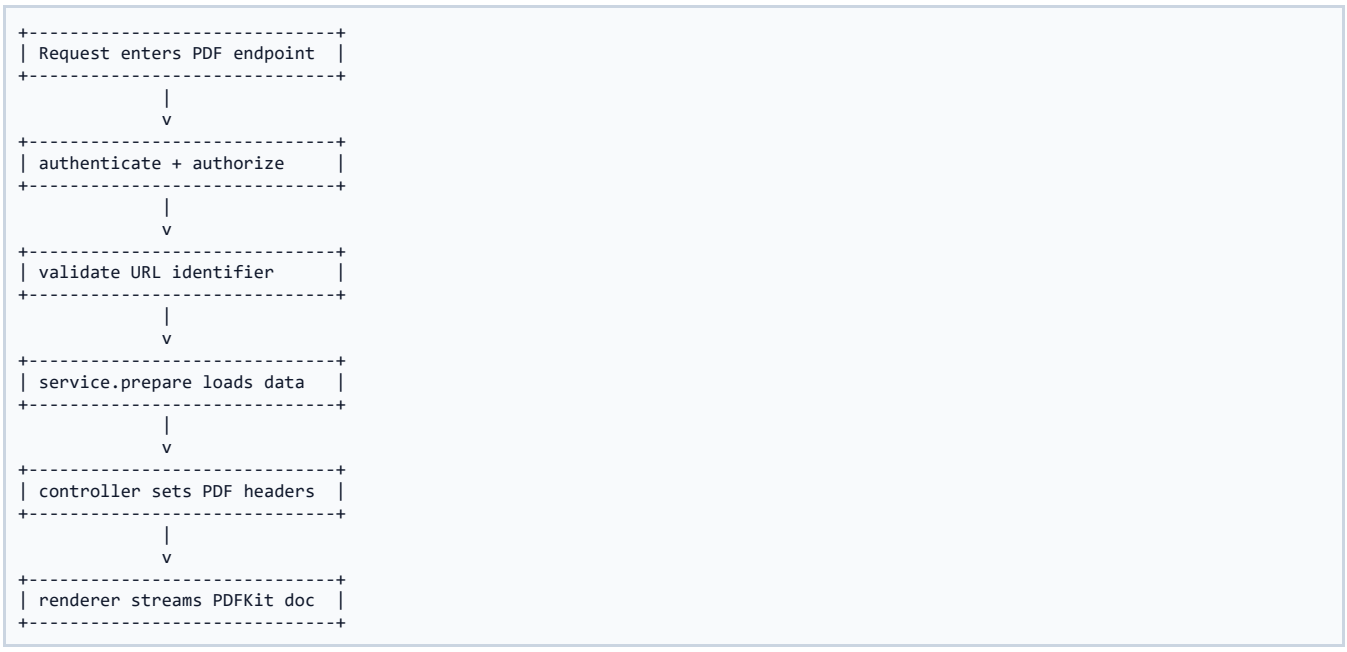
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.

Reason: preserve sealed data rules, role-aware access, and predictable document generation.

Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram



Step 4 - Backend PDF Module Creation

Description

A new src/modules/pdf module was introduced. It contains validators, repository loaders, service prepare functions, controllers, routes, shared template utilities, and three PDF templates. This isolated PDF logic from existing job card and job request modules.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 4 - Backend PDF Module Creation.

Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.

Reason: preserve sealed data rules, role-aware access, and predictable document generation.

Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Step 5 - Repository Read Models

Description

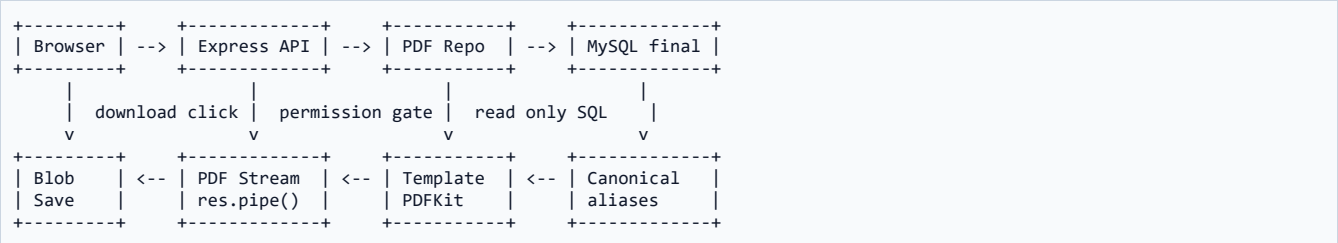
pdf.repo.js became the only file that knows real database columns. It loads the full Job Card payload and the full Job Request payload. All fields are aliased into canonical names so renderers stay clean and do not depend on legacy SQL names.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 5 - Repository Read Models.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram



Step 6 - Shared Header Strategy

Description

The templates share _isroHeader.js for official-style SAC/TIMCD header, typography, colors, page number stamping, and NULL-safe formatters. This avoids three separate implementations of the same official visual identity.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 6 - Shared Header Strategy.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Step 7 - Certificate Renderer

Description

jobCardCertificate.js renders the locked single-page certificate. It is strict and does not add pages. If accessories overflow the allowed space, the certificate shows a compact summary while the full details PDF carries the complete data.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 7 - Certificate Renderer.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Step 8 - Job Card Details Renderer

Description

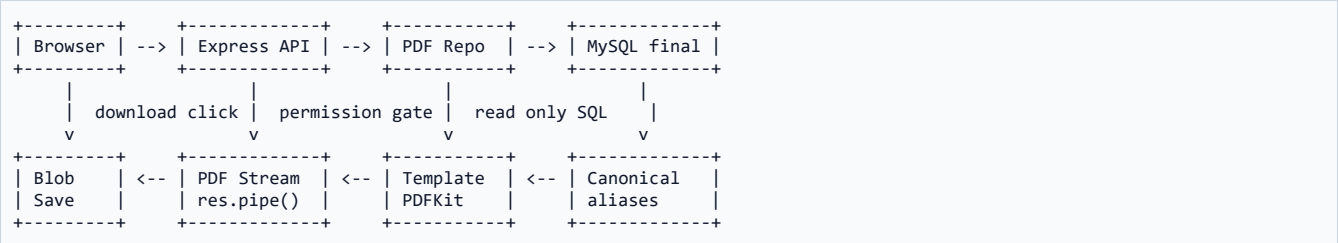
jobCardDetails.js renders a multi-page internal PDF with sections A through O. It consolidates equipment, accessories, submitted/received data, maintenance, equipment used, awaiting data, spares, contract/warranty, observations, checklist, documents, completion, closure, and status history.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 8 - Job Card Details Renderer.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram



Step 9 - Job Request Details Renderer

Description

jobRequestDetails.js renders a multi-page request archive. It captures classification, equipment snapshot, division/project, submitter, workflow actors, linked job card, accessories, and status transition history.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 9 - Job Request Details Renderer.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Step 10 - Two-Phase Service Pattern

Description

The service does all loading, existence checks, row-scope checks, and eligibility checks before any PDF byte is emitted. The controller sets headers only after prepare succeeds. This prevents partial PDF streams from replacing JSON error envelopes.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 10 - Two-Phase Service Pattern.

Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.

Reason: preserve sealed data rules, role-aware access, and predictable document generation.

Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Step 11 - Routes and Mounting

Description

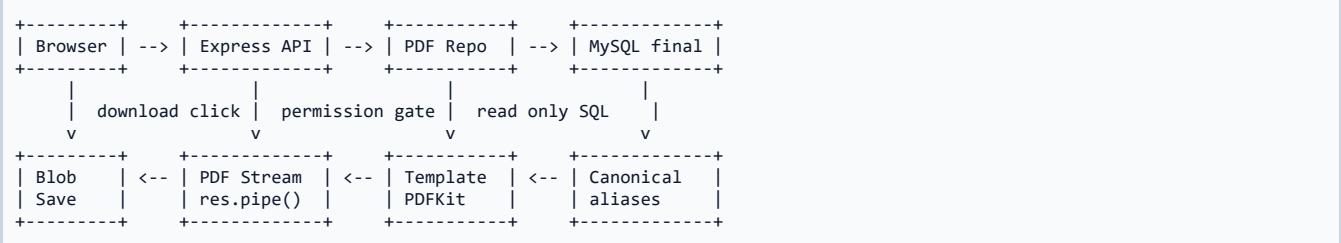
The PDF routers are mounted under existing job-cards and job-requests roots. They are mounted before the main job card/job request routers so .pdf paths are claimed correctly and do not collide with generic :id routes.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 11 - Routes and Mounting.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram



Step 12 - Frontend API Wrapper

Description

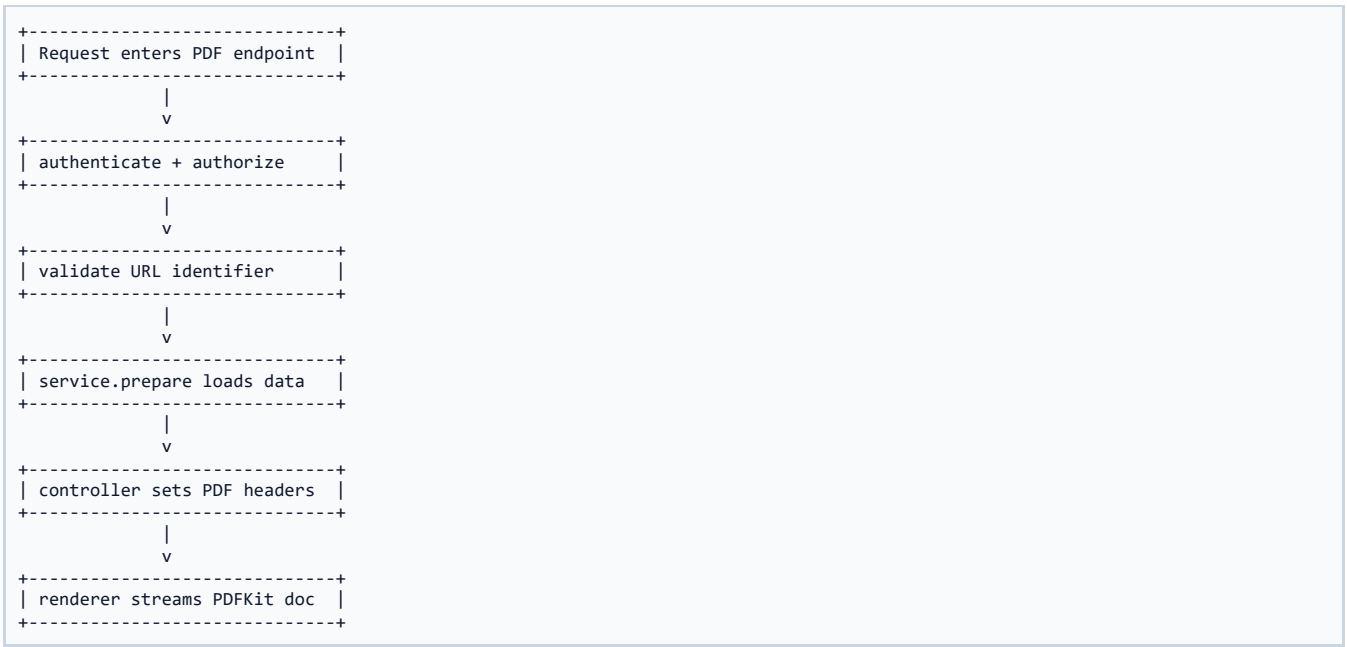
src/lib/api/pdf.js performs axios blob downloads, extracts filenames from Content-Disposition, triggers browser download, and unwraps JSON errors from Blob responses so toasts can show readable messages.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 12 - Frontend API Wrapper.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram



Step 13 - Job Card Header Integration

Description

The Job Card Detail header now has a live Download Report button for the certificate and a Download Full Details button. The certificate button is status-gated and permission-gated in the UI, but the backend is still the final authority.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 13 - Job Card Header Integration.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Step 14 - Job Request Header Integration

Description

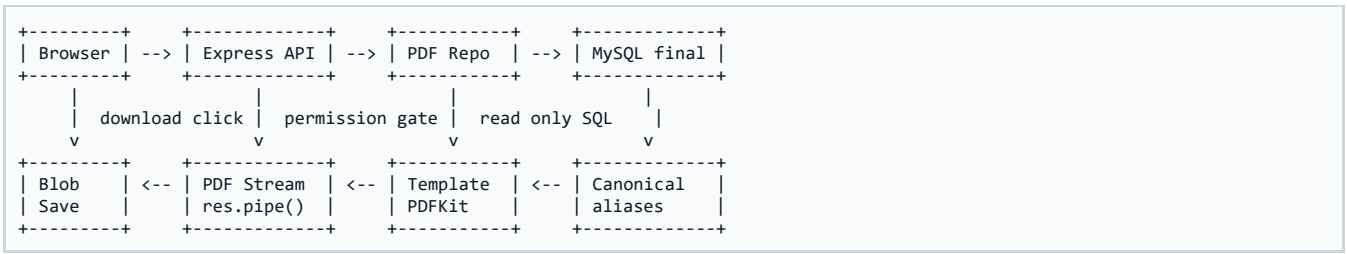
The Job Request Detail header gained Download Request PDF. It is permission-gated in UI and row-scoped in backend. Normal Users can download their own request PDF without gaining broad visibility.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 14 - Job Request Header Integration.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram



Step 15 - Smoke Matrix

Description

The smoke script validated unauthenticated 401, valid PDF 200s, ineligible 409, bogus 404s, RBAC, Normal User row scope, filename patterns, PDF page counts, PDF magic bytes, zero-write proof, and repo aliasing.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 15 - Smoke Matrix.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Step 16 - Pagination Bug Discovery

Description

The first multi-page renderer produced 75 pages for a small Job Card due to pageAdded header rendering plus PDFKit auto-pagination. This was caught by page count inspection and fixed before sealing.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 16 - Pagination Bug Discovery.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Step 17 - Pagination Fix

Description

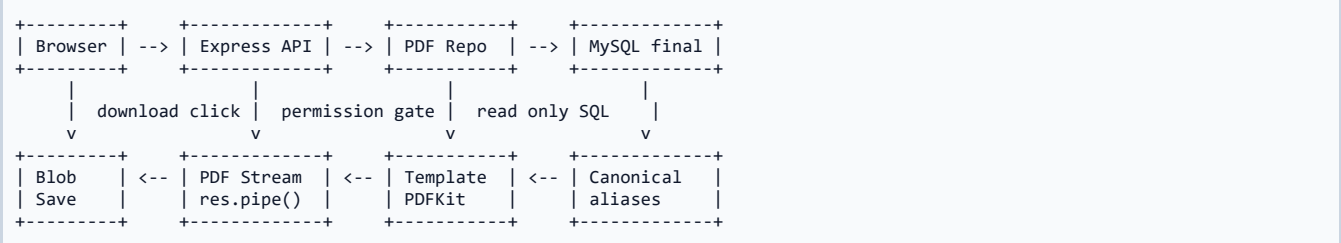
The fix removed re-entrant pageAdded header binding and enforced flow-disciplined gridKV rendering with fixed row heights, value height caps, ellipsis, and explicit ensureRoomFor calls.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 17 - Pagination Fix.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram



Step 18 - Final Green Result

Description

After the fix, the Certificate was 1 page, Job Card Details was 12 pages, Job Request Details was 6 pages, and the smoke matrix reported 23 passed / 0 failed.

DSRC - Decision / Structure / Reason / Consequence

Decision: Step 18 - Final Green Result.
Structure: implemented as part of the Phase 11 read-only PDF generation pipeline.
Reason: preserve sealed data rules, role-aware access, and predictable document generation.
Consequence: a future developer can extend PDFs without breaking workflow transitions or database integrity.

Layer	Phase 11 Responsibility	Senior Check
Database	Read from existing schema or seed permissions additively	No destructive DDL
Backend	Validate, authorize, load, prepare, stream	No hidden workflow write
Template	Render official/internal PDF structure	No null leakage, no overflow
Frontend	Expose download action cleanly	Blob download + user feedback
QA	Prove endpoint, RBAC, and zero-write behaviour	Smoke matrix green

Diagram

Permission Grant Matrix

Description

This matrix is the operational contract of Phase 11. It distinguishes formal calibration certificate access from broad details access. View-only users can see details but cannot download the formal certificate.

Permission	SA	LIC	LE	NU	VO
job_card:download-certificate	Yes	Yes	Yes	No	No
job_card:download-details	Yes	Yes	Yes	No	Yes
job_request:download-details	Yes	Yes	Yes	Yes	Yes

DS Senior Note

A matrix page is not decoration. It transforms implementation into a contract. A future engineer can compare the system against this table to confirm no accidental permission drift or route drift has occurred.

Endpoint Matrix

Description

Routes were attached under existing module roots, not under a separate /pdf namespace. This keeps URLs meaningful and colocated with the domain objects.

Endpoint	Permission	Result
GET /api/v1/job-cards/:id/certificate.pdf	job_card:download-certificate	PDF #1 certificate or 409
GET /api/v1/job-cards/:id/details.pdf	job_card:download-details	PDF #2 full JC details
GET /api/v1/job-requests/:id/details.pdf	job_request:download-details	PDF #3 JR details

DS Senior Note

A matrix page is not decoration. It transforms implementation into a contract. A future engineer can compare the system against this table to confirm no accidental permission drift or route drift has occurred.

Smoke Verification Matrix

Description

The verification matrix is the sealing proof. It checks the feature as a user would use it and as an auditor would inspect it.

Check	Expected	Outcome
No-auth access	401 on all PDF routes	PASS
VC certificate	200, application/pdf, 1 page	PASS
Assigned certificate	409 CONFLICT	PASS
JC details	200, multi-page PDF	PASS
JR details	200, multi-page PDF	PASS
Zero-write audit_log	Before == after	PASS
Repo aliasing	No legacy tokens outside repo	PASS

DS Senior Note

A matrix page is not decoration. It transforms implementation into a contract. A future engineer can compare the system against this table to confirm no accidental permission drift or route drift has occurred.

PDF Output Metrics

Description

The metrics prove that the final generation result is controlled. The earlier 75-page anomaly was resolved before sealing.

Document	Example Result	Interpretation
Certificate	1 page, 4.7 KB	Single-page lock successful
Job Card Details	12 pages, 12.8 KB	Pagination bug fixed
Job Request Details	6 pages, 6.4 KB	Lifecycle archive complete

DS Senior Note

A matrix page is not decoration. It transforms implementation into a contract. A future engineer can compare the system against this table to confirm no accidental permission drift or route drift has occurred.

Module Deep Dive 1: 410__phase11_pdf_permissions.sql

Module Purpose

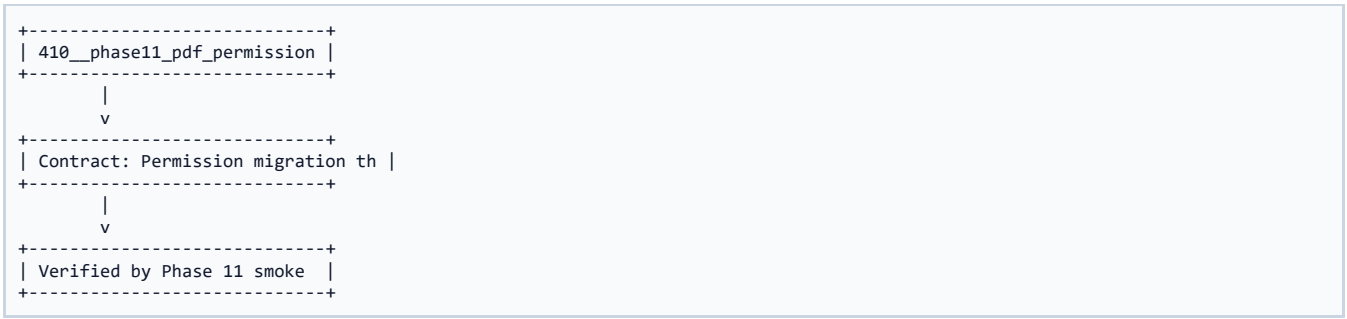
Permission migration that seeds three granular PDF permissions using an idempotent ADD-only style.

Concern	Implementation Meaning
Boundary	410__phase11_pdf_permissions.sql has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep 410__phase11_pdf_permissions.sql focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 2: pdf.validators.js

Module Purpose

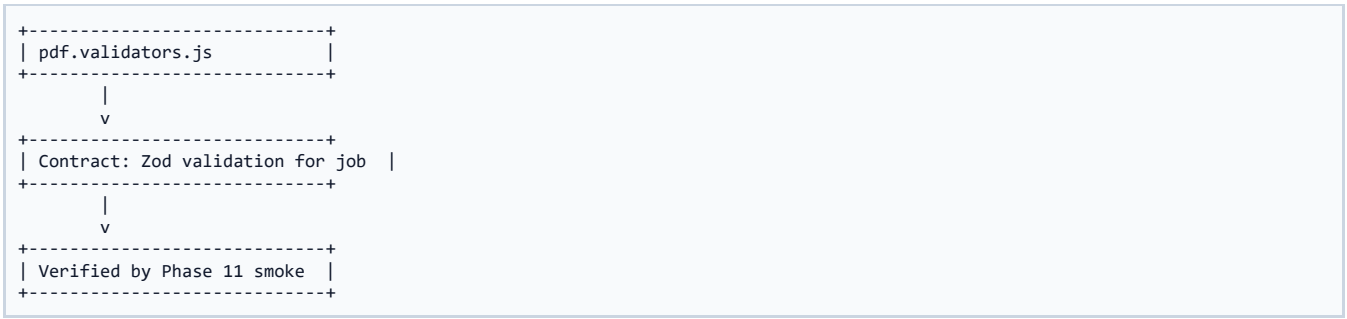
Zod validation for job card section job numbers and job request numeric IDs.

Concern	Implementation Meaning
Boundary	pdf.validators.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep pdf.validators.js focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 3: pdf.repo.js

Module Purpose

Read-only repository that loads full Job Card and Job Request payloads from MySQL and aliases legacy columns.

Concern	Implementation Meaning
Boundary	pdf.repo.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

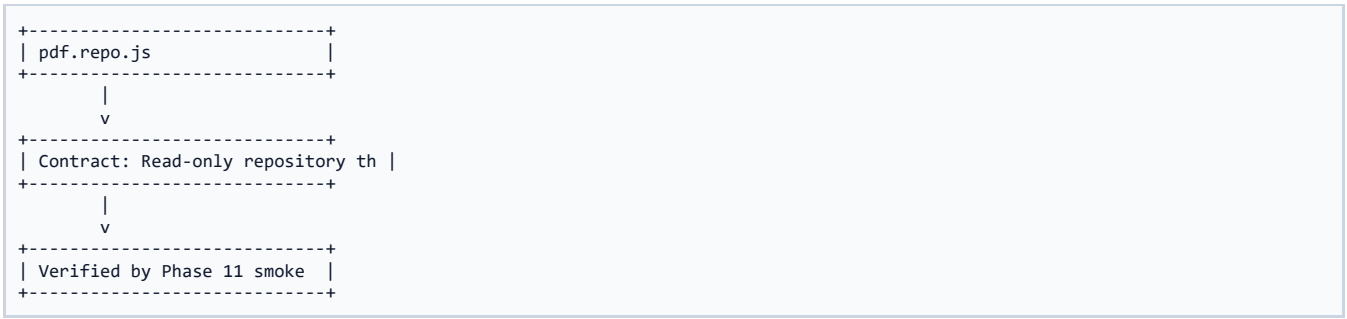
Decision: keep pdf.repo.js focused.

Reason: focus prevents hidden coupling.

Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.

Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 4: _isroHeader.js

Module Purpose

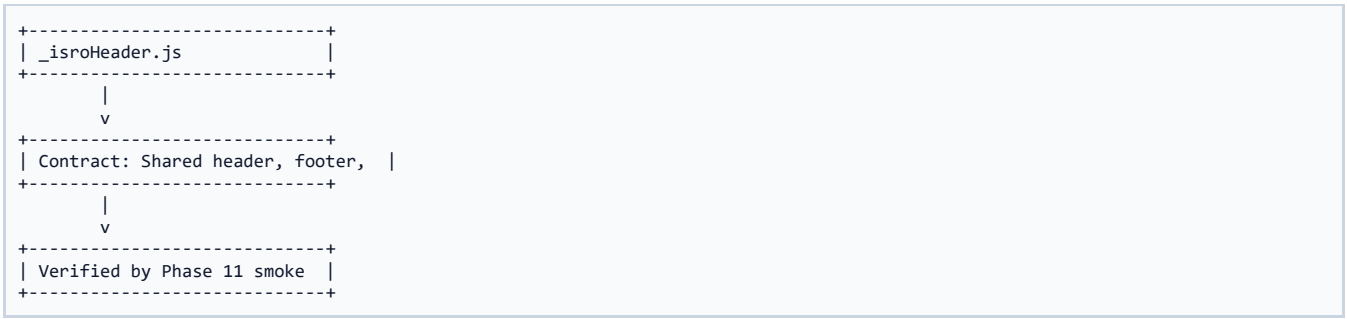
Shared header, footer, color palette, page numbering, logo fallback, and formatting helper utilities.

Concern	Implementation Meaning
Boundary	_isroHeader.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep _isroHeader.js focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 5: jobCardCertificate.js

Module Purpose

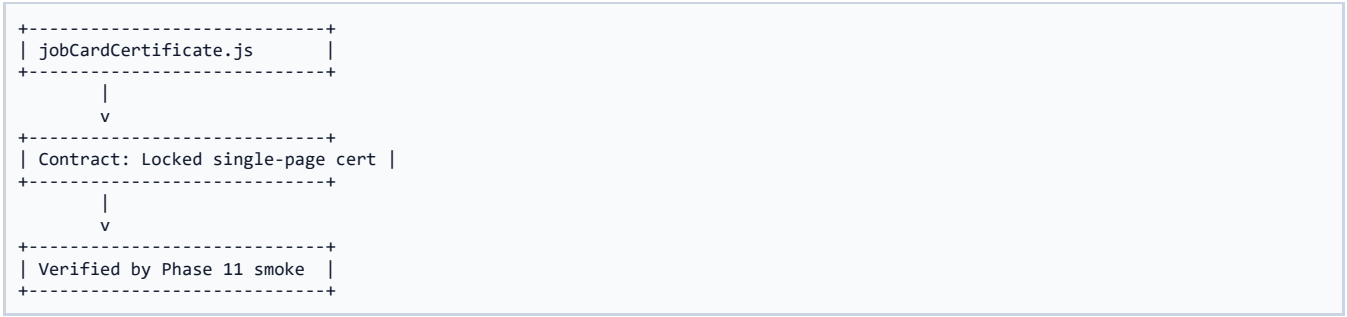
Locked single-page certificate renderer matching TIMCD form structure.

Concern	Implementation Meaning
Boundary	jobCardCertificate.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep jobCardCertificate.js focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 6: jobCardDetails.js

Module Purpose

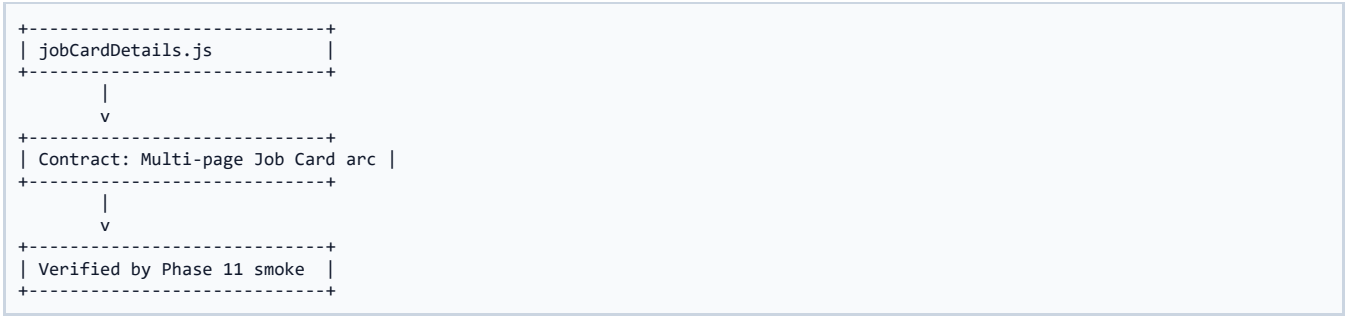
Multi-page Job Card archive renderer covering all major tabs and Phase 9 child tables.

Concern	Implementation Meaning
Boundary	jobCardDetails.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep jobCardDetails.js focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 7: jobRequestDetails.js

Module Purpose

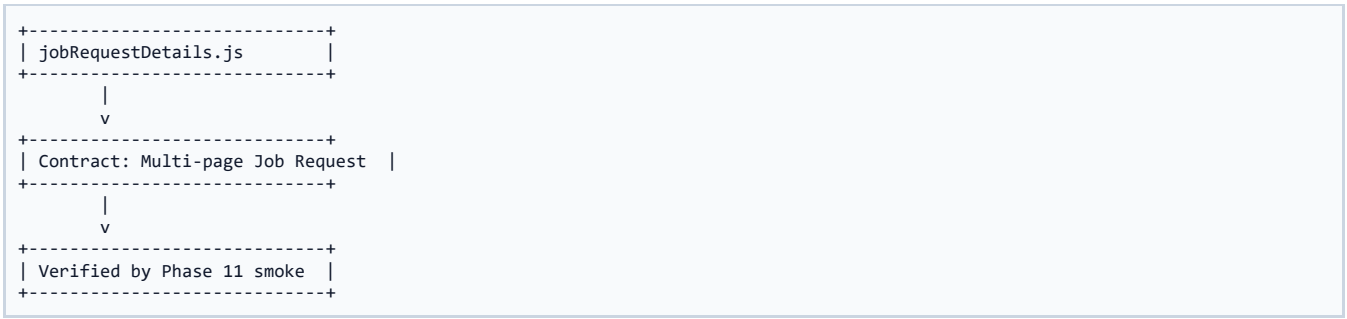
Multi-page Job Request archive renderer covering request lifecycle and linked Job Card.

Concern	Implementation Meaning
Boundary	jobRequestDetails.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep jobRequestDetails.js focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 8: pdf.service.js

Module Purpose

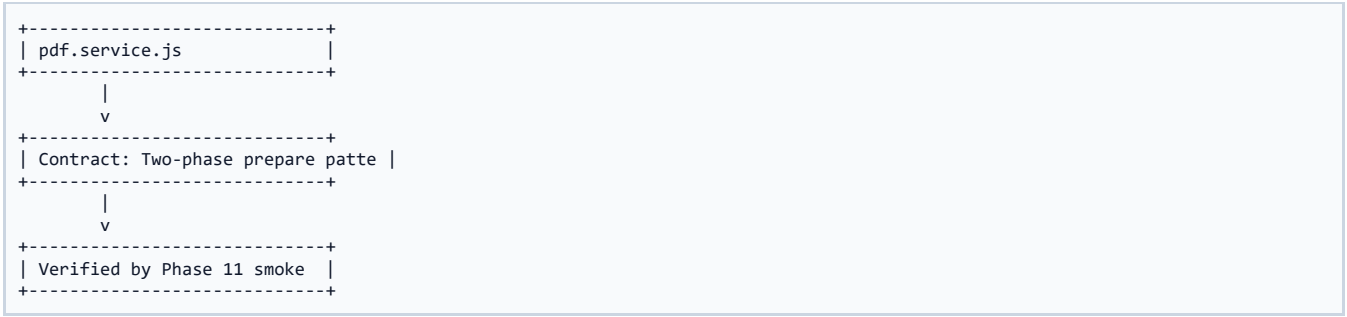
Two-phase prepare pattern that performs checks before streaming bytes.

Concern	Implementation Meaning
Boundary	pdf.service.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep pdf.service.js focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 9: pdf.controller.js

Module Purpose

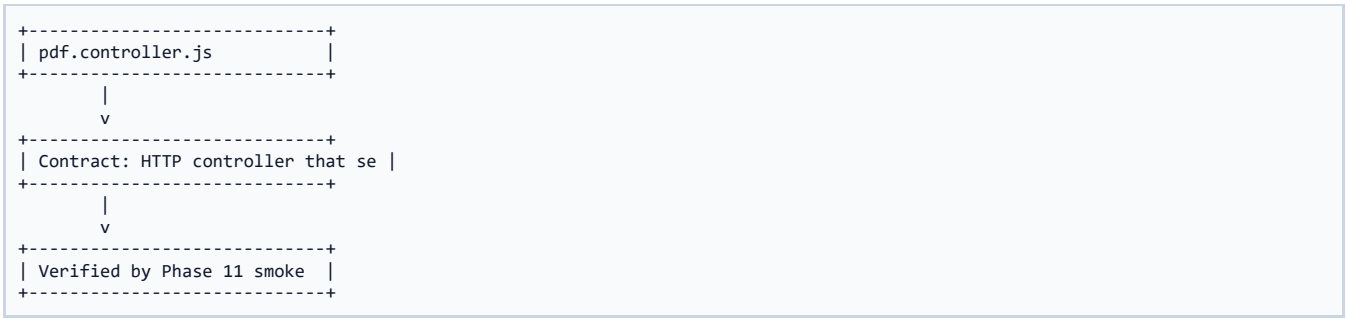
HTTP controller that sets PDF headers and invokes prepared renderer closures.

Concern	Implementation Meaning
Boundary	pdf.controller.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep pdf.controller.js focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 10: pdf.routes.js

Module Purpose

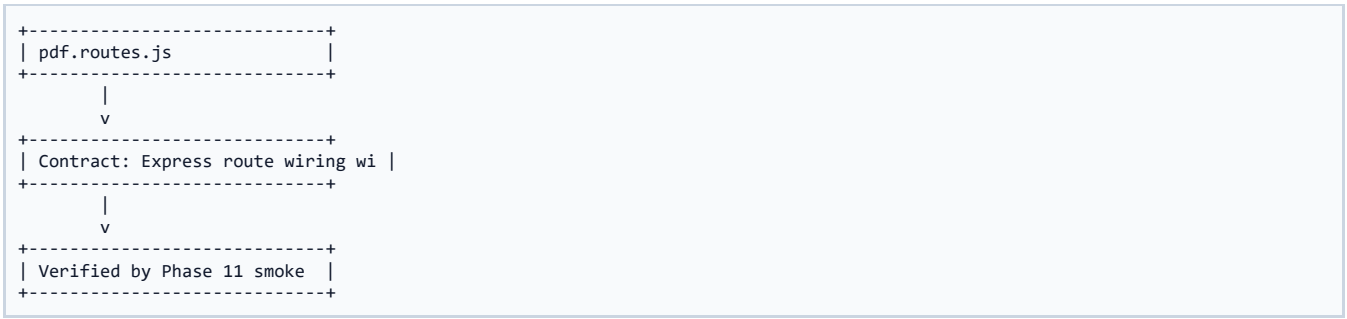
Express route wiring with authentication, authorization, validation, and controller attachment.

Concern	Implementation Meaning
Boundary	pdf.routes.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep pdf.routes.js focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 11: src/server.js mount

Module Purpose

Mounts PDF routers before domain routers so .pdf paths win route matching.

Concern	Implementation Meaning
Boundary	src/server.js mount has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

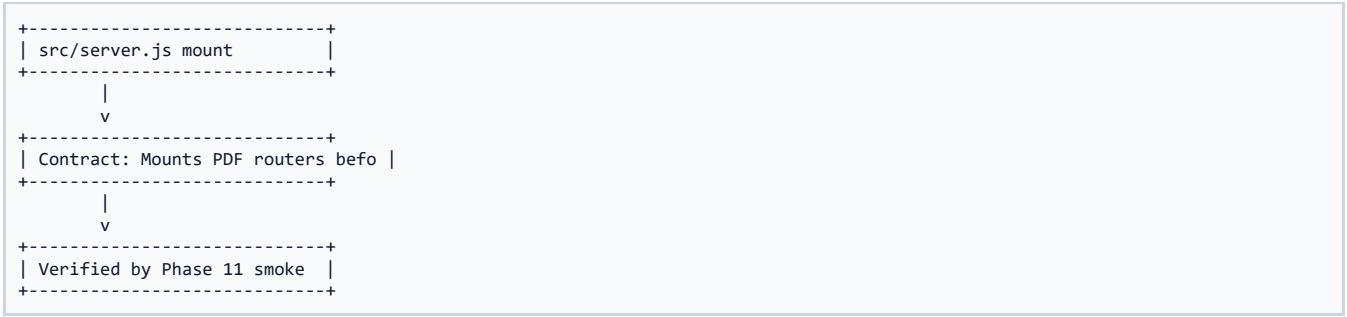
Decision: keep src/server.js mount focused.

Reason: focus prevents hidden coupling.

Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.

Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 12: src/lib/api/pdf.js

Module Purpose

Frontend Blob download wrapper with filename extraction and JSON-error unwrap.

Concern	Implementation Meaning
Boundary	src/lib/api/pdf.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

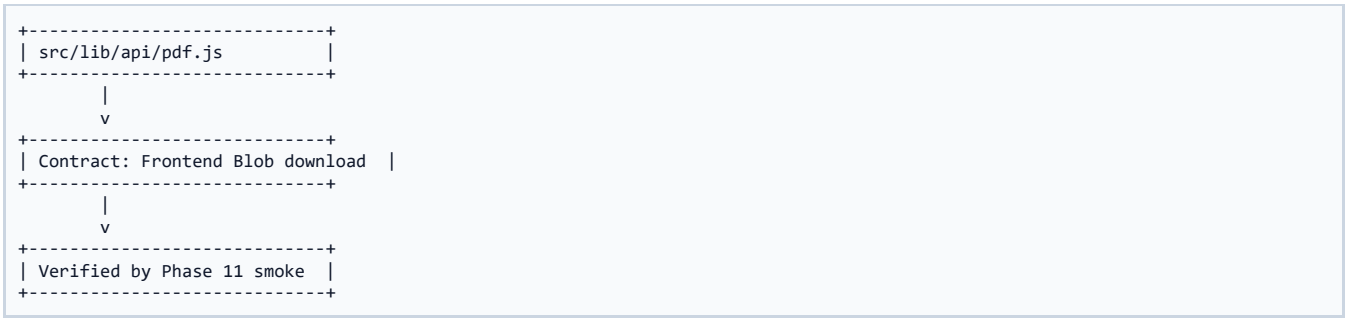
Decision: keep src/lib/api/pdf.js focused.

Reason: focus prevents hidden coupling.

Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.

Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 13: JobCards DetailHeader.jsx

Module Purpose

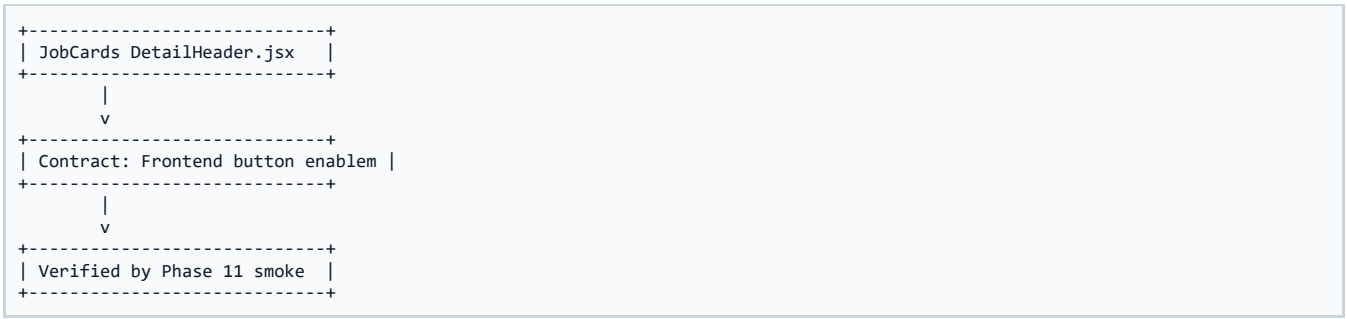
Frontend button enablement for Download Report and Download Full Details.

Concern	Implementation Meaning
Boundary	JobCards DetailHeader.jsx has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep JobCards DetailHeader.jsx focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 14: JobRequests DetailHeader.jsx

Module Purpose

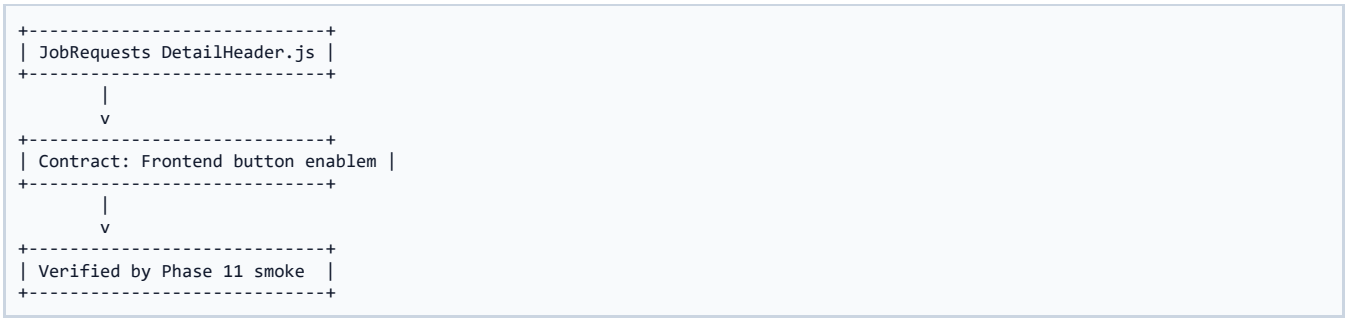
Frontend button enablement for Download Request PDF.

Concern	Implementation Meaning
Boundary	JobRequests DetailHeader.jsx has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep JobRequests DetailHeader.jsx focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



Module Deep Dive 15: smoke_phase11.js

Module Purpose

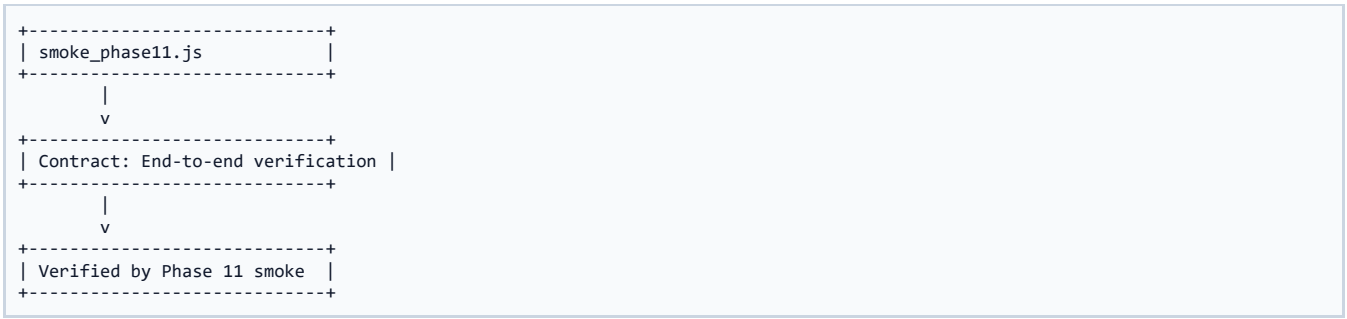
End-to-end verification script for PDFs, RBAC, row scope, errors, filenames, and zero writes.

Concern	Implementation Meaning
Boundary	smoke_phase11.js has a specific layer responsibility; it should not absorb unrelated module logic.
Inputs	Uses either route params, authenticated actor, repository payload, or UI state depending on layer.
Outputs	Emits either canonical data, PDF stream, route response, or browser download.
Failure mode	Returns explicit JSON errors before streaming or disabled UI with readable toast feedback.
Sealed check	Must remain compatible with Phase 7, Phase 9, and Phase 10 sealed doctrines.

DSRC Block

Decision: keep smoke_phase11.js focused.
Reason: focus prevents hidden coupling.
Structure: isolate validation, data loading, render logic, response streaming, and UI downloading.
Consequence: future PDF changes can be made in one layer without breaking the others.

Layer Map



P11-D1 - Certificate layout locked

Locked Decision

The certificate must remain faithful to the supplied reference and must not be redesigned casually.

Dimension	Locked Explanation
Decision	Certificate layout locked
Why	The certificate must remain faithful to the supplied reference and must not be redesigned casually.
Risk prevented	Workflow ambiguity, permission drift, layout drift, or accidental data mutation
Future rule	Do not change without reopening Phase 11 formally

Consequence

This decision changes how future development must be approached. It is not a temporary note; it is a sealed engineering constraint for the next modules and handover.

P11-D2 - Certificate eligibility

Locked Decision

Only COMPLETED and VERIFIED_CLOSED job cards can generate the formal certificate.

Dimension	Locked Explanation
Decision	Certificate eligibility
Why	Only COMPLETED and VERIFIED_CLOSED job cards can generate the formal certificate.
Risk prevented	Workflow ambiguity, permission drift, layout drift, or accidental data mutation
Future rule	Do not change without reopening Phase 11 formally

Consequence

This decision changes how future development must be approached. It is not a temporary note; it is a sealed engineering constraint for the next modules and handover.

P11-D3 - Multi-page details

Locked Decision

JC details and JR details may be multi-page because their purpose is internal completeness, not one-page certificate compactness.

Dimension	Locked Explanation
Decision	Multi-page details
Why	JC details and JR details may be multi-page because their purpose is internal completeness, not one-page certificate compactness.
Risk prevented	Workflow ambiguity, permission drift, layout drift, or accidental data mutation
Future rule	Do not change without reopening Phase 11 formally

Consequence

This decision changes how future development must be approached. It is not a temporary note; it is a sealed engineering constraint for the next modules and handover.

P11-D4 - Zero persistence

Locked Decision

PDFs are streamed and never persisted to disk or DB.

Dimension	Locked Explanation
Decision	Zero persistence
Why	PDFs are streamed and never persisted to disk or DB.
Risk prevented	Workflow ambiguity, permission drift, layout drift, or accidental data mutation
Future rule	Do not change without reopening Phase 11 formally

Consequence

This decision changes how future development must be approached. It is not a temporary note; it is a sealed engineering constraint for the next modules and handover.

P11-D5 - Pagination discipline

Locked Decision

Avoid re-entrant pageAdded header rendering; use explicit flow control and fixed-height rows.

Dimension	Locked Explanation
Decision	Pagination discipline
Why	Avoid re-entrant pageAdded header rendering; use explicit flow control and fixed-height rows.
Risk prevented	Workflow ambiguity, permission drift, layout drift, or accidental data mutation
Future rule	Do not change without reopening Phase 11 formally

Consequence

This decision changes how future development must be approached. It is not a temporary note; it is a sealed engineering constraint for the next modules and handover.

P11-D6 - Permission naming convention

Locked Decision

Use job_card:* and job_request:* convention instead of shorthand.

Dimension	Locked Explanation
Decision	Permission naming convention
Why	Use job_card:* and job_request:* convention instead of shorthand.
Risk prevented	Workflow ambiguity, permission drift, layout drift, or accidental data mutation
Future rule	Do not change without reopening Phase 11 formally

Consequence

This decision changes how future development must be approached. It is not a temporary note; it is a sealed engineering constraint for the next modules and handover.

P11-D7 - JR row scope

Locked Decision

Normal User foreign JR PDF downloads collapse to 404, not 403.

Dimension	Locked Explanation
Decision	JR row scope
Why	Normal User foreign JR PDF downloads collapse to 404, not 403.
Risk prevented	Workflow ambiguity, permission drift, layout drift, or accidental data mutation
Future rule	Do not change without reopening Phase 11 formally

Consequence

This decision changes how future development must be approached. It is not a temporary note; it is a sealed engineering constraint for the next modules and handover.

P11-D8 - Logo strategy

Locked Decision

Use image logos when present, typographic fallback when absent; ship no logo binaries by default.

Dimension	Locked Explanation
Decision	Logo strategy
Why	Use image logos when present, typographic fallback when absent; ship no logo binaries by default.
Risk prevented	Workflow ambiguity, permission drift, layout drift, or accidental data mutation
Future rule	Do not change without reopening Phase 11 formally

Consequence

This decision changes how future development must be approached. It is not a temporary note; it is a sealed engineering constraint for the next modules and handover.

Code Appendix - Guided Extracts

How to read this appendix

The appendix is not random source dumping. Each code extract is included because it captures a design rule: permission seeding, validation, read-only repository logic, template rendering, two-phase service preparation, HTTP streaming, frontend Blob download, or smoke verification.

Code Appendix: DATABASE\phase3\migrations\410__phase11_pdf_permissions.sql

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	102
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from DATABASE\phase3\migrations\410__phase11_pdf_permissions.sql around line 1

```
-- ═══════════════════════════════════════════════════════════════
-- CMCMIS_SIMPLIFIED — Migration 410 (Phase 11 PDF Generation)
-- File:     410__phase11_pdf_permissions.sql
-- Purpose:  Idempotent ADD-only seed for the 3 NEW PDF download
--           permissions used by Phase 11. Follows the same additive
--           seed pattern as mig 122 (Phase 8) + 400 (Phase 10).
--
--           NOTE on naming: existing Phase-3 permissions use the
--           `job_card:*` / `job_request:*` snake_case-with-colon
--           convention (e.g. `job_card:read-detail`, `job_card:
--           generate-pdf`, `job_request:approve`). Phase 11 follows
--           the SAME convention for consistency — the Phase 11 prompt's
--           hyphenated `jobcard:*` shorthand is treated as the same
--           concept, just spelled in the project's permanent style.
--
-- NEW PERMISSIONS (3):
--   job_card:download-certificate     — PDF #1 (LOCKED layout, single page,
--                                       only for status COMPLETED / VERIFIED_CLOSED)
--   job_card:download-details         — PDF #2 (full multi-page details)
--   job_request:download-details      — PDF #3 (multi-page JR details)
--
-- GRANT MATRIX (Phase 11 §3, senior call on VIEW_ONLY):
--   SUPER_ADMIN     → all 3
--   LAB_IN_CHARGE   → all 3
--   LAB_ENGINEER    → certificate + JC details + JR details (all 3)
--   NORMAL_USER     → JR details only (own-scope handled at row-level)
--   VIEW_ONLY       → JC details + JR details (NO certificate — internal
--                     calibration document; org default deny)
--
-- LEGACY: `job_card:generate-pdf` (Phase 3) is retained for backwards
-- compatibility on the legacy `/job-cards/:id/pdf` stub. New code uses
-- the granular permissions above.
--
-- IDEMPOTENT: INSERT IGNORE on permissions.uk_perm_code and
--             role_permissions PK (role_id, permission_id).
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from DATABASE\phase3\migrations\410__phase11_pdf_permissions.sql around line 34

```
--             role_permissions PK (role_id, permission_id).
-- ============================================================


SET NAMES utf8mb4;

-- ── 410.1  Permission codes (no-op if already present) ──────────────
INSERT IGNORE INTO `permissions`
  (`permission_code`,                 `resource`,    `action`,              `description`,
VALUES
  ('job_card:download-certificate',    'job_card',    'download-certificate','Download Job Card Certificate PDF (locked single-
page TIMCD layout)',
  ('job_card:download-details',        'job_card',    'download-details',    'Download full multi-page Job Card Details PDF
(every tab)',
  ('job_request:download-details',     'job_request', 'download-details',    'Download multi-page Job Request Details PDF',



-- ── 410.2  Grant matrix per Phase 11 §3 ─────────────────────────────
-- Resolve role_ids by role_code (Phase 3 sealed): SUPER_ADMIN=1,
-- LAB_IN_CHARGE=2, LAB_ENGINEER=3, NORMAL_USER=4, VIEW_ONLY=5.
-- INSERT IGNORE on the (role_id, permission_id) PK makes re-runs no-ops.

-- SUPER_ADMIN + LAB_IN_CHARGE + LAB_ENGINEER → all 3 PDF perms.
INSERT IGNORE INTO `role_permissions` (`role_id`, `permission_id`, `granted_at`, `granted_by`)
SELECT r.role_id, p.permission_id, NOW(6), 'BOOTSTRAP'
  FROM `roles` r
  CROSS JOIN `permissions` p
 WHERE r.role_code IN ('SUPER_ADMIN', 'LAB_IN_CHARGE', 'LAB_ENGINEER')
   AND p.permission_code IN (
        'job_card:download-certificate',
        'job_card:download-details',
        'job_request:download-details'
       );

-- NORMAL_USER → JR details only (own-scope governed by row-level scope).
INSERT IGNORE INTO `role_permissions` (`role_id`, `permission_id`, `granted_at`, `granted_by`)
SELECT r.role_id, p.permission_id, NOW(6), 'BOOTSTRAP'
  FROM `roles` r
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from DATABASE\phase3\migrations\410__phase11_pdf_permissions.sql around line 68

```
SELECT r.role_id, p.permission_id, NOW(6), 'BOOTSTRAP'
  FROM `roles` r
  CROSS JOIN `permissions` p
 WHERE r.role_code = 'NORMAL_USER'
   AND p.permission_code = 'job_request:download-details';

-- VIEW_ONLY → JC details + JR details (NO certificate — senior decision).
INSERT IGNORE INTO `role_permissions` (`role_id`, `permission_id`, `granted_at`, `granted_by`)
SELECT r.role_id, p.permission_id, NOW(6), 'BOOTSTRAP'
  FROM `roles` r
  CROSS JOIN `permissions` p
 WHERE r.role_code = 'VIEW_ONLY'
   AND p.permission_code IN (
        'job_card:download-details',
        'job_request:download-details'
       );


-- ── 410.3  Verify ──────────────────────────────────────────────
SELECT
  p.permission_code,
  COUNT(rp.role_id)                           AS granted_roles,
  GROUP_CONCAT(r.role_code ORDER BY r.role_id) AS roles
  FROM `permissions` p
  LEFT JOIN `role_permissions` rp ON rp.permission_id = p.permission_id
  LEFT JOIN `roles`            r  ON r.role_id        = rp.role_id
 WHERE p.permission_code IN (
        'job_card:download-certificate',
        'job_card:download-details',
        'job_request:download-details'
       )
 GROUP BY p.permission_code
 ORDER BY p.permission_code;

SELECT '✓ Migration 410 complete — Phase 11 PDF permissions granted' AS result;
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\src\modules\pdf\pdf.validators.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	54
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\src\modules\pdf\pdf.validators.js around line 1

```
// ============================================================================
// src/modules/pdf/pdf.validators.js  —  Zod schemas for PDF endpoints
// ----------------------------------------------------------------------------
// PHASE 11 — PDF Generation
//
// All three PDF endpoints take their target identifier from the URL `:id`
// param. We validate the param shape here so the route can fail fast
// before hitting the repo:
//
//   /api/v1/job-cards/:id/certificate.pdf   — :id = JM_SectionJobNo (varchar(9), e.g. 'J00024219')
//   /api/v1/job-cards/:id/details.pdf       — :id = JM_SectionJobNo
//   /api/v1/job-requests/:id/details.pdf    — :id = JR_JOBREQUESTNO (positive int)
//
// SECURITY NOTES
//   • :id values flow into parameterised SQL via `?` placeholders, so even
//     malicious input is bound, never interpolated. The regex below is
//     defence-in-depth, not the primary protection.
//   • Section job numbers in this codebase come in TWO flavours:
//       - new MVP format "J" + 8 digits (e.g. "J00024219")    — Phase 7 Slice 2
//       - legacy format like "62026043" or "01/24/001"        — pre-MVP rows
//     We accept letters, digits, and "/" so legacy rows can still be opened
//     in the UI. Max length 9 (legacy `JM_SectionJobNo VARCHAR(9)`).
// ============================================================================

'use strict';

const { z } = require('zod');

// ── Section Job No (job card identifier) ─────────────────────────────────
// VARCHAR(9). Allow uppercase letters, digits and forward slash (legacy
// rows like "01/24/001"). NO spaces, NO injection-friendly characters.
const sectionJobNoSchema = z.object({
  id: z
    .string()
    .trim()
    .min(1, 'Job card id is required')
    .max(9, 'Job card id must be ≤ 9 characters')
    .regex(/^[A-Za-z0-9/]+$/, 'Job card id must contain only letters, digits or /'),
}).strict();
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\src\modules\pdf\pdf.repo.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	391
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\src\modules\pdf\pdf.repo.js around line 1

```
// ============================================================================
// src/modules/pdf/pdf.repo.js  —  Read-only full-payload loaders for PDFs
// ----------------------------------------------------------------------------
// PHASE 11 — PDF Generation
//
// DOCTRINE
//   • ONLY this file mentions real legacy column names (JM_*, JR_*, EQM_*).
//   • Every SELECT aliases to canonical snake_case so the renderers never
//     see legacy column names.
//   • READ-ONLY. Zero INSERT/UPDATE/DELETE in this module. No audit rows,
//     no download logs (Phase 11 §1.F + §5).
//   • Every value bound via `?` placeholder. Column names are NEVER
//     interpolated from user input.
//
// FUNCTIONS
//   loadJobCardFull(sectionJobNo)
//     → master row + all 5 Phase-9 child tables + state history + parent JR
//       summary + equipment + division + employee-name resolutions.
//       Used by BOTH PDF #1 (Certificate) and PDF #2 (Details).
//
//   loadJobRequestFull(jrNo)
//     → JR detail row + division + employee names + linked JC summary +
//       accessories + state history. Used by PDF #3.
//
// CHILD TABLES (Phase 9, all keyed by `jc_section_no = JM_SectionJobNo`):
//   jc_maintenance_details
//   jc_spares_used
//   jc_task_checklist
//   jc_documents              (filter: deleted_at IS NULL)
//   jc_observations_readings
//   job_card_status_history
//
// JR child:
//   job_request_accessories   (keyed by jr_no = JR_JOBREQUESTNO)
//   job_request_status_history
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\pdf.repo.js around line 178

```
  // (multi-row maintenance + spares + readings) total wall-clock is one
  // round trip, not six.
  const [
    [maintenance],
    [spares],
    [tasks],
    [documents],
    [observations],
    [history],
    [parentAccessories],
  ] = await Promise.all([
    pool.query(
      `SELECT sr_no, defect_description, observation, action_taken, remarks,
              created_at, updated_at
         FROM jc_maintenance_details
        WHERE jc_section_no = ?
        ORDER BY sr_no ASC, id ASC`,
      [sectionJobNo],
    ),
    pool.query(
      `SELECT sr_no, spare_type, source, part_no, part_description,
              quantity, cost, created_at, updated_at
         FROM jc_spares_used
        WHERE jc_section_no = ?
        ORDER BY sr_no ASC, id ASC`,
      [sectionJobNo],
    ),
    pool.query(
      `SELECT task_text, is_custom, is_completed, completed_at,
              completed_by_employee_id, order_index
         FROM jc_task_checklist
        WHERE jc_section_no = ?
        ORDER BY order_index ASC, id ASC`,
      [sectionJobNo],
    ),
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\pdf.repo.js around line 357

```
    [accessories],
    [history],
  ] = await Promise.all([
    pool.query(
      `SELECT accessory_type AS type, accessory_name AS name,
              serial_no, position
         FROM job_request_accessories
        WHERE jr_no = ?
        ORDER BY position ASC, acc_id ASC`,
      [jrNo],
    ),
    pool.query(
      `SELECT from_status, to_status, transitioned_at,
              transitioned_by, reason
         FROM job_request_status_history
        WHERE jr_no = ?
        ORDER BY transitioned_at ASC, history_id ASC`,
      [jrNo],
    ),
  ]);

  return {
    ...main,
    children: {
      accessories,
      history,
    },
  };
}


module.exports = {
  loadJobCardFull,
  loadJobRequestFull,
};
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\src\modules\pdf\templates_isroHeader.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	239
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\src\modules\pdf\templates_isroHeader.js around line 1

```
// ============================================================================
// src/modules/pdf/templates/_isroHeader.js  —  Shared header + footer helpers
// ----------------------------------------------------------------------------
// PHASE 11 — PDF Generation
//
// PURPOSE
//   Single source of truth for the ISRO + SAC seal blocks and the
//   "SPACE APPLICATIONS CENTRE / TIMCD" title bar that appears at the
//   top of every CMCMIS PDF (Certificate, JC Details, JR Details).
//
//   Also exports a `stampPageNumbers()` helper used by the multi-page
//   PDFs (#2 and #3) — the Certificate is strictly single-page so it
//   does NOT call this; the helper would still work (1 of 1) but the
//   certificate template renders its own footer-less single page.
//
// LOGO STRATEGY
//   This module looks for the official ISRO + SAC SVGs/PNGs at
//   `src/assets/isro-sac-logo.{svg,png}` and friends. If absent, it
//   falls back to a typographic "ISRO / SAC" seal block — exactly the
//   approach Phase-10 reports.pdf.js uses. This is intentional:
//     1. Government licensing concerns mean we ship no logo binary by
//        default; the CMCMIS instance operator drops the official assets
//        into `src/assets/` and they show up automatically.
//     2. The fallback is visually consistent with the supplied reference
//        PDF (typographic + framed) and prints cleanly in B/W.
//
//   To upgrade later: drop a 256×256 PNG at one of the searched paths;
//   `tryLoadLogo()` will pick it up on next request.
// ============================================================================

'use strict';

const fs   = require('fs');
const path = require('path');
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\templates_isroHeader.js around line 102

```
  const pageWidth = doc.page.width;
  const leftX  = PAGE_MARGIN_X;
  const rightX = pageWidth - PAGE_MARGIN_X;

  const startY = doc.y;
  const sealW = compact ? 52 : 60;
  const sealH = compact ? 52 : 60;

  // ── LEFT SEAL — ISRO ──────────────────────────────────────
  if (ISRO_LOGO_PATH) {
    try {
      doc.image(ISRO_LOGO_PATH, leftX, startY, { fit: [sealW, sealH], align: 'center', valign: 'center' });
    } catch {
      drawTypographicSeal(doc, leftX, startY, sealW, sealH, 'ISRO', '□□□□ · INDIA', 'Space Dept.');
    }
  } else {
    drawTypographicSeal(doc, leftX, startY, sealW, sealH, 'ISRO', '□□□□ · INDIA', 'Space Dept.');
  }

  // ── RIGHT SEAL — SAC ──────────────────────────────────────
  const sacX = rightX - sealW;
  if (SAC_LOGO_PATH) {
    try {
      doc.image(SAC_LOGO_PATH, sacX, startY, { fit: [sealW, sealH], align: 'center', valign: 'center' });
    } catch {
      drawTypographicSeal(doc, sacX, startY, sealW, sealH, 'SAC', 'Ahmedabad', 'CMCMIS');
    }
  } else {
    drawTypographicSeal(doc, sacX, startY, sealW, sealH, 'SAC', 'Ahmedabad', 'CMCMIS');
  }

  // ── CENTRE TITLE BLOCK ────────────────────────────────────
  const centreX = leftX + sealW + 12;
  const centreW = sacX - 12 - centreX;
  const titleFont = compact ? 13 : 14;
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\templates_isroHeader.js around line 205

```
  const dd = String(dt.getUTCDate()).padStart(2, '0');
  const mm = String(dt.getUTCMonth() + 1).padStart(2, '0');
  const yy = dt.getUTCFullYear();
  return `${dd}-${mm}-${yy}`;
}

function fmtDateTime(d, fallback = '—') {
  if (d === null || d === undefined || d === '') return fallback;
  const dt = (d instanceof Date) ? d : new Date(d);
  if (Number.isNaN(dt.getTime())) return fallback;
  return `${fmtDate(dt)} ${String(dt.getUTCHours()).padStart(2, '0')}:${String(dt.getUTCMinutes()).padStart(2, '0')}`;
}

function fmtText(s, fallback = '—') {
  if (s === null || s === undefined) return fallback;
  const str = String(s).trim();
  return str === '' ? fallback : str;
}


module.exports = {
  // Layout constants
  PAGE_MARGIN_X,
  PAGE_MARGIN_TOP,
  PAGE_MARGIN_BOTTOM,
  FOOTER_HEIGHT,
  COLORS,
  // Renderers
  drawHeader,
  stampPageNumbers,
  // NULL-safe formatters
  fmtDate,
  fmtDateTime,
  fmtText,
};
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\src\modules\pdf\templates\jobCardCertificate.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	391
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\src\modules\pdf\templates\jobCardCertificate.js around line 1

```
// ============================================================================
// src/modules/pdf/templates/jobCardCertificate.js
// ----------------------------------------------------------------------------
// PDF #1 — JOB CARD CERTIFICATE (LOCKED LAYOUT — DO NOT REDESIGN)
//
// PHASE 11 — PDF Generation
//
// Replicates the supplied reference document
// `JobCard_cmcmis_simplified_DSMG.pdf` exactly:
//
//    ┌──────────────────────────────────────────────────────────────────┐
//    │ ISRO    SPACE APPLICATIONS CENTRE / TIMCD / JOB REQUEST     SAC  │
//    │ logo    OF T&ME FOR CALIBRATION                            seal │
//    ├──────────────────────────────────────────────────────────────────┤
//    │ Equipment ID | Job Card No. | Date                               │
//    ├──────────────────────────────────────────────────────────────────┤
//    │ 1. EQUIPMENT IDENTIFICATION                                      │
//    │    Name · Make · Model · Serial · Main Frame · Options           │
//    │    [Accessory rows table: Item Type | Sr.No. | Item Name | …]    │
//    ├──────────────────────────────────────────────────────────────────┤
//    │ 2. REQUEST CLASSIFICATION                                        │
//    │    Sent after repairs: ☐ Yes ☐ No · Remarks: ___                 │
//    ├──────────────────────────────────────────────────────────────────┤
//    │ 3. USER SUBMISSION & FORWARDING                                  │
//    │    Submitted By · Forwarded Through HOD/EIC                      │
//    ├──────────────────────────────────────────────────────────────────┤
//    │ 4. INSTRUCTIONS FOR USER (static text)                           │
//    ├──────────────────────────────────────────────────────────────────┤
//    │ 5. FOR TIMCD USE — internal receiving/planning/assignment        │
//    ├──────────────────────────────────────────────────────────────────┤
//    │ 6. CALIBRATION LAB RECEIPT                                       │
//    ├──────────────────────────────────────────────────────────────────┤
//    │ 7. CONFORMITY, USER SIGNATURE & REVIEW                           │
//    └──────────────────────────────────────────────────────────────────┘
//
//
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\templates\jobCardCertificate.js around line 178

```
     .text(`${fmtText(payload.equipment_type, '')}-${fmtText(payload.equipment_id, '')}`,
           stripX + 8, idStripY + 12, { width: colW - 16, lineBreak: false });
  // Job Card No.
  doc.font('Helvetica-Bold').fontSize(8)
     .text('Job Card No.:', stripX + 8 + colW, idStripY + 4, { lineBreak: false });
  doc.font('Helvetica').fontSize(9)
     .text(jcCode(payload), stripX + 8 + colW, idStripY + 12, { width: colW - 16, lineBreak: false });
  // Date (use jc_recd_date — the JC's received date, per template)
  doc.font('Helvetica-Bold').fontSize(8)
     .text('Date:', stripX + 8 + colW * 2, idStripY + 4, { lineBreak: false });
  doc.font('Helvetica').fontSize(9)
     .text(fmtDate(payload.jc_recd_date), stripX + 8 + colW * 2, idStripY + 12,
           { width: colW - 16, lineBreak: false });
  doc.y = idStripY + idStripH + 4;

  // ── SECTION 1 — EQUIPMENT IDENTIFICATION ──────────────────────
  sectionBanner(doc, 1, 'EQUIPMENT IDENTIFICATION', 'AUTO-FILLED FROM EQUIPMENT MASTER / REQUEST');
  const s1y = doc.y + 4;
  const halfW = (stripW - 12) / 2;
  kv(doc, stripX,            s1y,      halfW, 'Name of Equipment',     payload.equipment_name);
  kv(doc, stripX + halfW + 12, s1y,    halfW, 'Make',                  payload.equipment_make);
  kv(doc, stripX,            s1y + 24, halfW, 'Identification / Model No.', payload.equipment_model_no ||
payload.equipment_mfg_model_name);
  kv(doc, stripX + halfW + 12, s1y + 24, halfW,'Serial No.',           payload.equipment_serial_no);
  kv(doc, stripX,            s1y + 48, halfW, 'Main Frame',           payload.equipment_mfg_model_name);
  kv(doc, stripX + halfW + 12, s1y + 48, halfW,'Option(s) & Description', payload.equipment_option_desc);
  doc.y = s1y + 72;

  // Accessory table — 2 rows (header + up to 2 data rows)
  const accRows = buildAccessoryRows(payload);
  drawAccessoryTable(doc, stripX, doc.y, stripW, accRows);
  doc.y += 50;            // table block is ~ 50 pt tall

  // ── SECTION 2 — REQUEST CLASSIFICATION ────────────────────────
  sectionBanner(doc, 2, 'REQUEST CLASSIFICATION', 'REPAIR / CALIBRATION CONTEXT');
  const s2y = doc.y + 4;
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\templates\jobCardCertificate.js around line 357

```
  // Header band
  doc.rect(x, y, w, headerH).fillAndStroke(COLORS.subtle, COLORS.border);
  doc.fillColor(COLORS.title).font('Helvetica-Bold').fontSize(7.5);
  let cx = x;
  headers.forEach((h, i) => {
    doc.text(h, cx + 4, y + 3, { width: widths[i] - 8, lineBreak: false });
    cx += widths[i];
  });

  // Data rows — always render 2 rows (blank if no data, for the print form)
  doc.font('Helvetica').fontSize(8.5).fillColor(COLORS.title);
  for (let r = 0; r < totalRows; r++) {
    const ry = y + headerH + r * rowH;
    doc.lineWidth(0.4).strokeColor(COLORS.border).rect(x, ry, w, rowH).stroke();
    cx = x;
    const data = rows[r];
    if (data && !data.overflow) {
      const cells = [data.type, data.sr_no, data.name, data.model_no, data.serial];
      cells.forEach((c, i) => {
        doc.text(fmtText(c, ''), cx + 4, ry + 3, { width: widths[i] - 8, lineBreak: false, ellipsis: true });
        cx += widths[i];
      });
    } else if (data && data.overflow) {
      doc.fillColor(COLORS.inkSoft).font('Helvetica-Oblique')
         .text(`… +${data.overflow} more accessories (see Full Details PDF)`,
               x + 4, ry + 3, { width: w - 8, lineBreak: false });
      doc.fillColor(COLORS.title).font('Helvetica');
    }
  }
}


module.exports = {
  renderJobCardCertificate,
};
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\src\modules\pdf\templates\jobCardDetails.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	452
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\src\modules\pdf\templates\jobCardDetails.js around line 1

```
// ============================================================================
// src/modules/pdf/templates/jobCardDetails.js
// ----------------------------------------------------------------------------
// PDF #2 — JOB CARD DETAILS (own design, multi-page)
//
// PHASE 11 — PDF Generation
//
// Renders the COMPLETE Job Card payload — every tab — as a single
// professional internal document. Used for archival / hand-over so a
// supervisor can read the full state of one card in one PDF without
// opening the web UI.
//
// LAYOUT
//   Page 1 starts with the shared ISRO + SAC + TIMCD header (compact),
//   then a JC-summary band (JC code, Section JobNo, Status, Priority,
//   Generated On, Generated By).
//
//   Body sections (each with a banner row + content block):
//     A. Equipment & Engineer
//     B. Accessories (parent JR + JC plug-in text)
//     C. Submitted & Received
//     D. Job Card Details (instrument received / job type / repair type / remarks)
//     E. Maintenance Details (multi-row table from jc_maintenance_details)
//     F. Equipments Used (free text)
//     G. Awaiting Information
//     H. Spares Used (multi-row table from jc_spares_used)
//     I. Contract / Warranty
//     J. Observations (free text + jc_observations_readings table)
//     K. Task Checklist (jc_task_checklist with done/total)
//     L. Documents (jc_documents — METADATA ONLY, no file bytes)
//     M. Completion
//     N. Closure
//     O. State History (job_card_status_history)
//
// PAGINATION
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\templates\jobCardDetails.js around line 209

```
}


// ── MAIN RENDERER ──────────────────────────────────────────────

function renderJobCardDetails(payload, stream, meta = {}) {
  const doc = new PDFDocument({
    size: 'A4',
    layout: 'portrait',
    margin: PAGE_MARGIN_X,
    bufferPages: true,
    autoFirstPage: true,
  });
  doc.pipe(stream);

  // ── Re-draw the header on every new page ──────────────────────
  // PDFKit emits 'pageAdded' for every page AFTER the first; we
  // explicitly call drawHeader on the first page too.
  const drawTopBlock = () => {
    drawHeader(doc, {
      title: 'JOB CARD — DETAILS REPORT',
      subtitle: 'CMCMIS · Internal Document',
      compact: true,
    });
  };
  doc.on('pageAdded', drawTopBlock);
  drawTopBlock();

  // ── Summary band ──────────────────────────────────────────────
  const code = jcCode(payload);
  const summary = [
    { label: 'Job Card Code',  value: code },
    { label: 'Section Job No', value: payload.section_job_no },
    { label: 'Parent JR',      value: jrCode(payload) },
    { label: 'Status',         value: payload.status },
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\templates\jobCardDetails.js around line 418

```
  sectionBanner(doc, 'N', 'Closure');
  gridKV(doc, [
    { label: 'Reviewed By',                      value: payload.reviewed_by },
    { label: 'Review Date',                      value: fmtDate(payload.review_date) },
    { label: 'Equipment Received By Customer',   value: payload.equipment_received_by_customer },
    { label: 'Customer Received Date',           value: fmtDate(payload.customer_received_date) },
    { label: 'Customer Acknowledged',            value: payload.customer_acknowledged ? 'Yes' : 'No' },
    { label: 'Verified-Closed By',               value: payload.verified_closed_by_employee_id
        ? `${payload.verified_closed_by_name || ''} (${payload.verified_closed_by_employee_id})`
        : '—' },
    { label: 'Verified-Closed At',               value: fmtDateTime(payload.verified_closed_at) },
    { label: 'Reopen Count',                     value: payload.reopen_count ?? 0 },
  ], 2);
  blockText(doc, 'Review Comments',     payload.review_comments);
  blockText(doc, 'Final Closure Notes', payload.final_closure_notes);

  // ── O. State History ──────────────────────────────────────
  sectionBanner(doc, 'O', `State Transition History (${(payload.children?.history || []).length} entries)`);
  drawTable(doc, null, [
    { header: 'From',          key: 'from_status',     width: 100 },
    { header: 'To',            key: 'to_status',       width: 100 },
    { header: 'When',          key: 'transitioned_at', width: 130, format: fmtDateTime },
    { header: 'By',            key: 'transitioned_by', width: 100 },
    { header: 'Reason',        key: 'reason',          width: 150 },
  ], payload.children?.history || []);

  // ── Stamp page numbers + footer on every page ─────────────
  stampPageNumbers(doc, { docId: code });
  doc.end();
}


module.exports = {
  renderJobCardDetails,
};
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\src\modules\pdf\templates\jobRequestDetails.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	314
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\src\modules\pdf\templates\jobRequestDetails.js around line 1

```
// ============================================================================
// src/modules/pdf/templates/jobRequestDetails.js
// ----------------------------------------------------------------------------
// PDF #3 — JOB REQUEST DETAILS (own design, multi-page)
//
// PHASE 11 — PDF Generation
//
// One professional document per Job Request showing its full lifecycle:
// classification, equipment snapshot, division, submitter snapshot,
// approval / rejection / engineer assignment, linked Job Card summary,
// accessories list, and the chronological status history.
//
// LAYOUT
//   Page 1: ISRO + SAC header → JR-summary band → sections A..G.
//   Multi-page allowed (history can be long; we auto-paginate with the
//   shared header re-drawn at the top of each page via 'pageAdded').
//
//   Sections:
//     A. Classification
//     B. Equipment Snapshot
//     C. Division & Project
//     D. Submitter
//     E. Workflow Actors (Approval / Rejection / Assigned Engineer)
//     F. Linked Job Card
//     G. Accessories
//     H. Status History
// ============================================================================

'use strict';

const PDFDocument = require('pdfkit');
const {
  PAGE_MARGIN_X,
  PAGE_MARGIN_BOTTOM,
  FOOTER_HEIGHT,
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\templates\jobRequestDetails.js around line 140

```
      const chars = String(v).length;
      const cpl = Math.max(8, Math.floor((widths[i] - 2 * PAD) / 4.5));
      const lines = Math.min(4, Math.max(1, Math.ceil(chars / cpl)));
      const h = 6 + lines * 10;
      if (h > needed) needed = h;
    });
    ensureRoomFor(doc, needed);
    const ry = doc.y;
    doc.lineWidth(0.3).strokeColor(COLORS.border).rect(x, ry, tableW, needed).stroke();
    cx = x;
    columns.forEach((c, i) => {
      const v = c.format ? c.format(row[c.key], row) : (row[c.key] ?? '');
      doc.text(String(v ?? ''), cx + PAD, ry + 3, {
        width: widths[i] - 2 * PAD,
        align: c.align || 'left',
        height: needed - 6,
        ellipsis: true,
      });
      cx += widths[i];
    });
    doc.y = ry + needed;
  });
  doc.y += 4;
}

function jrCode(main) {
  const year = main.submitted_at_legacy ? new Date(main.submitted_at_legacy).getUTCFullYear()
             : main.created_at ? new Date(main.created_at).getUTCFullYear()
             : '----';
  return `JR-${year}-${String(main.jr_no || '').padStart(4, '0')}`;
}


// ── MAIN RENDERER ──────────────────────────────────────────────
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\templates\jobRequestDetails.js around line 280

```
      { label: 'JC Created At',        value: fmtDateTime(payload.linked_job_card_created_at) },
    ], 2);
  } else {
    doc.font('Helvetica-Oblique').fontSize(9).fillColor(COLORS.inkSoft);
    doc.text('No Job Card linked to this request yet.', PAGE_MARGIN_X + 4, doc.y);
    doc.y += 14;
  }

  // ── G. Accessories ──────────────────────────────────────
  sectionBanner(doc, 'G', `Accessories (${(payload.children?.accessories || []).length} entries)`);
  drawTable(doc, null, [
    { header: '#',         key: 'position',  width: 30, align: 'right' },
    { header: 'Type',      key: 'type',      width: 110 },
    { header: 'Name',      key: 'name',      width: 220 },
    { header: 'Serial No.',key: 'serial_no', width: 140 },
  ], payload.children?.accessories || []);

  // ── H. Status History ──────────────────────────────────
  sectionBanner(doc, 'H', `Status Transition History (${(payload.children?.history || []).length} entries)`);
  drawTable(doc, null, [
    { header: 'From',          key: 'from_status',     width: 100 },
    { header: 'To',            key: 'to_status',       width: 100 },
    { header: 'When',          key: 'transitioned_at', width: 130, format: fmtDateTime },
    { header: 'By',            key: 'transitioned_by', width: 100 },
    { header: 'Reason',        key: 'reason',          width: 170 },
  ], payload.children?.history || []);

  stampPageNumbers(doc, { docId: code });
  doc.end();
}


module.exports = {
  renderJobRequestDetails,
};
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\src\modules\pdf\pdf.service.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	124
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\src\modules\pdf\pdf.service.js around line 1

```
// ============================================================================
// src/modules/pdf/pdf.service.js  —  PDF assembly + eligibility checks
// ----------------------------------------------------------------------------
// PHASE 11 — PDF Generation
//
// Thin facade between the controller and the renderers. Does ONE business
// thing of its own: certificate eligibility — only Job Cards whose status
// is COMPLETED or VERIFIED_CLOSED can be downloaded as the locked
// certificate. Every other PDF is permission-gated only.
//
// CONTRACT
//   • Throws errors.notFound when the id doesn't exist.
//   • Throws errors.conflict (409) when the certificate is requested for
//     an ineligible status.
//   • Returns void from the render-* helpers — they pipe directly into the
//     Writable passed in (typically Express `res`).
//
// NO CACHING. Each request fetches fresh from MySQL. Per Phase-11 §5.
// ============================================================================

'use strict';

const repo = require('./pdf.repo');
const { errors } = require('../../middleware/errorHandler');

const { renderJobCardCertificate } = require('./templates/jobCardCertificate');
const { renderJobCardDetails }     = require('./templates/jobCardDetails');
const { renderJobRequestDetails }  = require('./templates/jobRequestDetails');

// Certificate is reserved for "this work is done" states only.
const CERT_ELIGIBLE = new Set(['COMPLETED', 'VERIFIED_CLOSED']);

// ── Helpers ──────────────────────────────────────────────────────────

/** Build the canonical JC code used in the filename. */
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\pdf.service.js around line 45

```
            : '0000';
  return `JR-${yr}-${String(payload.jr_no || '').padStart(4, '0')}`;
}


// ── Certificate (PDF #1) ──────────────────────────────────────────

/**
 * Generate the JOB CARD CERTIFICATE PDF and stream it.
 *
 * @param {string} sectionJobNo
 * @param {Object} actor   { employee_id, name, role, permissions }
 * @param {Writable} stream
 * @returns {Promise<{ filename: string }>}
 */
async function streamJobCardCertificate(sectionJobNo, actor, stream) {
  const payload = await repo.loadJobCardFull(sectionJobNo);
  if (!payload) throw errors.notFound(`Job Card not found: ${sectionJobNo}`);

  // Eligibility gate — single point of truth (§2 PDF #1).
  if (!CERT_ELIGIBLE.has(payload.status)) {
    throw errors.conflict(
      `Certificate is available only for COMPLETED or VERIFIED_CLOSED job cards (current: ${payload.status})`,
      { current_status: payload.status, eligible: [...CERT_ELIGIBLE] },
    );
  }

  const code = jcCode(payload);
  renderJobCardCertificate(payload, stream, { generated_by: actor });
  return { filename: `${code}_certificate.pdf` };
}


// ── JC Details (PDF #2) ──────────────────────────────────────────
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\pdf.service.js around line 90

```
// ── JR Details (PDF #3) ──────────────────────────────────────

/**
 * @param {number} jrNo
 * @param {Object} actor
 * @param {Writable} stream
 * @param {Object} [rowScope]  Optional row-level scope for Normal users.
 *                             { canReadAll: bool, ownerEmployeeId: string }
 */
async function streamJobRequestDetails(jrNo, actor, stream, rowScope) {
  const payload = await repo.loadJobRequestFull(jrNo);
  if (!payload) throw errors.notFound(`Job Request not found: ${jrNo}`);

  // Row-level scope: Normal Users can only download their OWN JR's PDF.
  // We collapse this into a 404 (not 403) so we don't leak the existence
  // of foreign IDs — mirrors Phase-7 Slice-2 JR Detail behaviour.
  if (rowScope && !rowScope.canReadAll
      && payload.submitted_by_employee_id !== rowScope.ownerEmployeeId) {
    throw errors.notFound(`Job Request not found: ${jrNo}`);
  }

  const code = jrCode(payload);
  renderJobRequestDetails(payload, stream, { generated_by: actor });
  return { filename: `${code}_details.pdf` };
}


module.exports = {
  streamJobCardCertificate,
  streamJobCardDetails,
  streamJobRequestDetails,
  // exported for tests:
  CERT_ELIGIBLE,
};
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\src\modules\pdf\pdf.controller.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	132
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\src\modules\pdf\pdf.controller.js around line 1

```
// ============================================================================
// src/modules/pdf/pdf.controller.js  —  HTTP handlers for 3 PDF endpoints
// ----------------------------------------------------------------------------
// PHASE 11 — PDF Generation
//
// Thin shims. Each handler:
//   1. Pulls the actor from req.user.
//   2. Calls into the service, which streams PDFKit chunks into res.
//   3. Sets Content-Type / Content-Disposition BEFORE the first chunk is
//      written (must precede pipe()).
//
// ERROR HANDLING NOTE
//   The service throws AppError (notFound / conflict) BEFORE the renderer
//   starts streaming — those errors flow through next(err) and end up in
//   errorHandler.js, which sends a JSON envelope. Once the renderer pipes
//   the first chunk we are committed (Content-Type is already pdf and
//   headers have been sent); any error inside the renderer goes to res's
//   error event handler below and just closes the connection — the client
//   browser shows a partial-download. This is fine in practice because
//   render-side errors are vanishingly rare (string formatting on already-
//   fetched data).
// ============================================================================

'use strict';

const service = require('./pdf.service');

function actorFromReq(req) {
  return {
    employee_id: req.user?.employeeId,
    name:        req.user?.fullName || req.user?.name || req.user?.employeeId || '',
    role:        req.user?.role || '',
    permissions: Array.isArray(req.user?.permissions) ? req.user.permissions : [],
  };
}
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\pdf.controller.js around line 49

```
 *  the returned `filename` value, which the controller has already used to
 *  set headers via this helper… actually we need a different approach
 *  because PDFKit streams synchronously after we return. See implementations
 *  below — they fetch the payload, set headers with the resolved filename,
 *  THEN call the renderer (zero double-fetch via service.* helpers that
 *  pre-resolve the filename). For simplicity, each handler below uses the
 *  service-resolved filename pattern via a small helper.
 */


// ── PDF #1 — JC Certificate ──────────────────────────────────────
async function jobCardCertificate(req, res, next) {
  try {
    // We need the filename BEFORE the renderer writes the first byte. We
    // do that by letting the service load the payload, derive the filename,
    // set the headers, then pipe. The service helper handles all that
    // internally because the renderer needs the payload anyway.
    //
    // Service.streamJobCardCertificate's contract: throws on not-found/409;
    // otherwise it writes headers via a callback we pass in, then streams.
    //
    // To keep the contract simple, we pre-fetch a tiny "head" query in the
    // controller via a temporary buffer wrapper. Actually the cleaner path:
    // let the service throw on 404/409 BEFORE writing any bytes; rely on
    // the fact that PDFKit doesn't actually emit until the renderer starts
    // — which it can't until the service finishes its repo call and
    // eligibility check. By then headers are still settable.
    //
    // The implementation below sets a placeholder filename and overwrites
    // before streaming via a small adapter:
    let resolvedFilename = `JC_${req.params.id}_certificate.pdf`;
    preparePdfHeaders(res, resolvedFilename);
    const { filename } = await service.streamJobCardCertificate(req.params.id, actorFromReq(req), res);
    // Update header AFTER (browsers ignore later header changes once
    // body bytes have begun, so this is a no-op for the client; the
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\src\modules\pdf\pdf.controller.js around line 98

```
    let resolvedFilename = `JC_${req.params.id}_details.pdf`;
    preparePdfHeaders(res, resolvedFilename);
    const { filename } = await service.streamJobCardDetails(req.params.id, actorFromReq(req), res);
    res.setHeader('X-CMCMIS-Filename', filename);
  } catch (e) {
    next(e);
  }
}

// ── PDF #3 — JR Details ──────────────────────────────────────
async function jobRequestDetails(req, res, next) {
  try {
    // Row-level scope: Normal Users (without job_request:read-all) see
    // only their own JRs. Mirrors the existing JR-detail policy.
    const perms = req.user?.permissions || [];
    const canReadAll = perms.includes('job_request:read-all');
    const rowScope = { canReadAll, ownerEmployeeId: req.user?.employeeId };

    let resolvedFilename = `JR_${req.params.id}_details.pdf`;
    preparePdfHeaders(res, resolvedFilename);
    const { filename } = await service.streamJobRequestDetails(
      req.params.id, actorFromReq(req), res, rowScope,
    );
    res.setHeader('X-CMCMIS-Filename', filename);
  } catch (e) {
    next(e);
  }
}


module.exports = {
  jobCardCertificate,
  jobCardDetails,
  jobRequestDetails,
};
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\src\modules\pdf\pdf.routes.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	73
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\src\modules\pdf\pdf.routes.js around line 1

```
// ============================================================================
// src/modules/pdf/pdf.routes.js  —  URL wiring
// ----------------------------------------------------------------------------
// PHASE 11 — PDF Generation
//
// The PDF routes attach to existing module path prefixes (job-cards and
// job-requests) rather than getting their own /pdf prefix. This keeps the
// URLs readable + lets the FE call them straight from the JC / JR detail
// pages without a new base path:
//
//   GET /api/v1/job-cards/:id/certificate.pdf    job_card:download-certificate
//   GET /api/v1/job-cards/:id/details.pdf        job_card:download-details
//   GET /api/v1/job-requests/:id/details.pdf     job_request:download-details
//
// Pipeline (per route):
//   authenticate → authorize(<perm>) → validate(:id) → controller
//
// We expose TWO separate Express routers (jobCardPdfRouter,
// jobRequestPdfRouter) so they can mount under the existing /job-cards
// and /job-requests roots respectively. Both export the same kind of
// router (no shared state).
// ============================================================================

'use strict';

const express = require('express');

const authenticate = require('../../middleware/authenticate');
const authorize    = require('../../middleware/authorize');
const validate     = require('../../middleware/validate');

const v    = require('./pdf.validators');
const ctrl = require('./pdf.controller');

// ── Job Card PDF endpoints (certificate + details) ──────────────────────
// mergeParams:true so the :id param flows through if mounted under a
// parent router that already declared :id (defensive; we mount at the
// /job-cards root via server.js so this is informational).
const jobCardPdfRouter = express.Router({ mergeParams: true });
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\db\discovery_inspect_pdf.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	26
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\db\discovery_inspect_pdf.js around line 1

```
// Quick inspector: list textual strings embedded in a PDFKit PDF.
'use strict';
const fs   = require('fs');
const path = require('path');
const os   = require('os');

const file = process.argv[2] || path.join(os.tmpdir(), 'cert.pdf');
const buf  = fs.readFileSync(file);
const s    = buf.toString('binary');

// PDFKit embeds text inside (...) tokens (PDF string literal). Capture
// printable runs ≥ 3 chars that contain a letter, ignoring control bytes.
const parenRe = /\(([^)\\]{2,80})\)/g;
const seen = new Set();
const out  = [];
let m;
while ((m = parenRe.exec(s)) !== null) {
  const t = m[1];
  if (!/[A-Za-z]{3,}/.test(t)) continue;
  if (/[\x00-\x1f]/.test(t))  continue;
  if (seen.has(t))             continue;
  seen.add(t);
  out.push(t);
}
out.slice(0, 60).forEach((t) => console.log(' ·', t));
console.log(`(${out.length} unique strings total)`);
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: FE\src\lib\api\pdf.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	105
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from FE\src\lib\api\pdf.js around line 1

```
// ============================================================================
// src/lib/api/pdf.js  —  PDF download HTTP wrappers
// ----------------------------------------------------------------------------
// PHASE 11 — PDF Generation
//
// Three thin axios wrappers. Each fires a GET with `responseType: 'blob'`,
// extracts the filename from Content-Disposition, and triggers a browser
// download via the canonical anchor-click pattern (same as the Phase-10
// reports / analytics download helpers).
//
// ERROR PROPAGATION
//   Axios throws on non-2xx. The caller catches and surfaces the error
//   via sonner toast. For the JSON-encoded error responses (404/409/403),
//   axios' default response handler tries to parse JSON — but since we
//   requested `blob`, the body is a Blob. The helpers below detect this
//   and re-read the Blob as text so the caller gets a parsed JSON error.
// ============================================================================

import { api } from '../api-client.js';

/** Trigger a browser download given a Blob + filename. */
function triggerBlobDownload(blob, filename) {
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  document.body.removeChild(a);
  setTimeout(() => URL.revokeObjectURL(url), 0);
}

/** Pull a sensible filename out of Content-Disposition, else fallback. */
function filenameFromHeaders(headers, fallback) {
  const cd = headers?.['content-disposition'] || '';
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from FE\src\lib\api\pdf.js around line 35

```
  const m  = cd.match(/filename="([^"]+)"/);
  return (m && m[1]) || fallback;
}

/**
 * Helper that does the blob fetch + error-blob unwrap.
 * On any non-2xx response, axios throws; if the error's response body
 * is a Blob (because of responseType:'blob'), we read it as JSON so the
 * UI can show the human-readable message and code.
 */
async function fetchPdfBlob(url, fallbackFilename) {
  try {
    const r = await api.get(url, { responseType: 'blob' });
    const filename = filenameFromHeaders(r.headers, fallbackFilename);
    triggerBlobDownload(r.data, filename);
    return { filename, bytes: r.data.size || 0 };
  } catch (err) {
    // Axios put the Blob in err.response.data. Read it as text + JSON-parse.
    const body = err?.response?.data;
    if (body instanceof Blob) {
      try {
        const text = await body.text();
        const j = JSON.parse(text);
        // Re-throw with the parsed envelope attached so the caller can show
        // it cleanly. We keep the original error chain (response.status etc.)
        // by mutating in place.
        err.response.data = j;
      } catch { /* leave as-is */ }
    }
    throw err;
  }
}


// ── PDF #1 — Job Card Certificate ──────────────────────────────
```

> **Reading Note**
>
> Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from FE\src\lib\api\pdf.js around line 71

```
/**
 * @param {string} sectionJobNo  e.g. "J00024219"
 * @returns {Promise<{filename:string, bytes:number}>}
 */
export function downloadJobCardCertificate(sectionJobNo) {
  return fetchPdfBlob(
    `/job-cards/${encodeURIComponent(sectionJobNo)}/certificate.pdf`,
    `${sectionJobNo}_certificate.pdf`,
  );
}

// ── PDF #2 — Job Card Full Details ──────────────────────────────
export function downloadJobCardDetails(sectionJobNo) {
  return fetchPdfBlob(
    `/job-cards/${encodeURIComponent(sectionJobNo)}/details.pdf`,
    `${sectionJobNo}_details.pdf`,
  );
}

// ── PDF #3 — Job Request Details ────────────────────────────────
export function downloadJobRequestDetails(jrNo) {
  return fetchPdfBlob(
    `/job-requests/${encodeURIComponent(jrNo)}/details.pdf`,
    `${jrNo}_details.pdf`,
  );
}


// ── Status helpers ──────────────────────────────────────────────

/** Certificate is only available for finished work. */
export const CERT_ELIGIBLE_STATUSES = Object.freeze(['COMPLETED', 'VERIFIED_CLOSED']);
export function isCertificateEligible(status) {
  return CERT_ELIGIBLE_STATUSES.includes(status);
}
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: FE\src\pages\jobCards\components\DetailHeader.jsx

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	179
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from FE\src\pages\jobCards\components\DetailHeader.jsx around line 1

```
// ============================================================================
// pages/jobCards/components/DetailHeader.jsx
// ----------------------------------------------------------------------------
// Top strip of the JC detail page: back-link, JC code + Section JobNo,
// equipment summary, status pill + priority pill, Download buttons.
//
// PHASE 11 UPDATE
//   The "Download Report" button (Certificate, PDF #1) is now WIRED.
//   It's enabled when:
//     - the user holds `job_card:download-certificate`, AND
//     - the JC's status ∈ { COMPLETED, VERIFIED_CLOSED }.
//   Otherwise we keep it disabled with a tooltip explaining why — the BE
//   independently re-validates (returns 409 on ineligible status, 403
//   on missing permission), so the FE gate is UX only.
//
//   A new "Download Full Details" button (PDF #2) appears next to it,
//   gated by `job_card:download-details`. Available for any JC the
//   user is already viewing.
// ============================================================================

import { useState } from 'react';
import { Link } from 'react-router-dom';
import { ArrowLeft, FileDown, FileText } from 'lucide-react';
import { toast } from 'sonner';

import { StatusPill } from '../../../components/StatusPill.jsx';
import { PriorityLabel } from '../../../components/PriorityLabel.jsx';
import { Button } from '../../../components/ui/Button.jsx';
import { useAuth } from '../../../lib/auth-context.jsx';
import {
  downloadJobCardCertificate,
  downloadJobCardDetails,
  isCertificateEligible,
} from '../../../lib/api/pdf.js';
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from FE\src\pages\jobCards\components\DetailHeader.jsx around line 72

```
async function onDownloadDetails() {
  setBusy('details');
  const id = toast.loading('Preparing Job Card Details PDF…');
  try {
    const { filename } = await downloadJobCardDetails(jc.section_job_no);
    toast.success(`Downloaded ${filename}`, { id });
  } catch (e) {
    const msg = e.response?.data?.error?.message
            || e.message
            || 'Failed to download details';
    toast.error(msg, { id });
  } finally {
    setBusy(null);
  }
}

return (
  <div className="space-y-3">
    <Link
      to="/job-cards"
      className="inline-flex items-center gap-1.5 text-sm text-ink-soft hover:text-ink"
    >
      <ArrowLeft size={14} strokeWidth={1.75} aria-hidden="true" />
      Back to Job Cards
    </Link>

    <div className="flex items-start justify-between gap-4">
      <div className="space-y-1">
        <h1 className="text-2xl font-semibold text-ink">
          Job Card: <span className="text-accent">{jc.card_code}</span>
        </h1>
        <p className="text-sm text-ink-soft">
          Section Job No: <span className="font-medium text-ink">{jc.section_job_no}</span>
          {jc.parent_jr_code ? (
            <>
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from FE\src\pages\jobCards\components\DetailHeader.jsx around line 145

```
              onClick={onDownloadCert}
            >
              <FileDown size={14} strokeWidth={1.75} aria-hidden="true" />
              {busy === 'cert' ? 'Preparing…' : 'Download Report'}
            </Button>
          </div>
        </div>
      </div>

      {/* Equipment summary strip */}
      <div className="bg-white border border-border rounded-lg p-3 grid grid-cols-1 md:grid-cols-4 gap-3 text-sm">
        <div>
          <div className="text-xs text-ink-soft">Equipment Name</div>
          <div className="text-ink font-medium truncate">{jc.equipment?.name || '—'}</div>
        </div>
        <div>
          <div className="text-xs text-ink-soft">Model No.</div>
          <div className="text-ink">{jc.equipment?.model_no || '—'}</div>
        </div>
        <div>
          <div className="text-xs text-ink-soft">Serial No.</div>
          <div className="text-ink">{jc.equipment?.serial_no || '—'}</div>
        </div>
        <div>
          <div className="text-xs text-ink-soft">Assigned Engineer</div>
          <div className="text-ink">
            {jc.assigned_engineer
              ? `${jc.assigned_engineer.name} (${jc.assigned_engineer.employee_id})`
              : '—'}
          </div>
        </div>
      </div>
    </div>
  );
}
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: FE\src\pages\jobRequests\components\DetailHeader.jsx

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	0
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from FE\src\pages\jobRequests\components\DetailHeader.jsx around line 1

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: FE\src\pages\jobRequests\components\DetailHeader.jsx

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	122
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from FE\src\pages\jobRequests\components\DetailHeader.jsx around line 1

```
// ============================================================================
// pages/jobRequests/components/DetailHeader.jsx
// ----------------------------------------------------------------------------
// Top strip of the JR Detail page: back-link, JR code, status pill, priority
// pill, type label, created/submitted timestamps.
//
// PHASE 11 UPDATE — adds "Download Request PDF" button (PDF #3).
// Gated by `job_request:download-details`. BE re-validates row-level scope
// (Normal Users see only their own JRs) and returns 404 for foreign IDs.
// ============================================================================

import { useState } from 'react';
import { Link } from 'react-router-dom';
import { ArrowLeft, FileDown } from 'lucide-react';
import { toast } from 'sonner';

import { StatusPill } from '../../../components/StatusPill.jsx';
import { PriorityLabel } from '../../../components/PriorityLabel.jsx';
import { Button } from '../../../components/ui/Button.jsx';
import { useAuth } from '../../../lib/auth-context.jsx';
import { downloadJobRequestDetails } from '../../../lib/api/pdf.js';

const JOB_TYPE_LABEL = {
  CALIBRATION:  'Calibration',
  REPAIR:       'Repair (Inspection)',
  REGISTRATION: 'Registration (Master Data)',
};

function fmt(iso) {
  if (!iso) return '—';
  try {
    return new Date(iso).toLocaleString(undefined, {
      year: 'numeric', month: 'short', day: '2-digit',
      hour: '2-digit', minute: '2-digit',
    });
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from FE\src\pages\jobRequests\components\DetailHeader.jsx around line 44

```
export function DetailHeader({ jr }) {
  const { user } = useAuth();
  const canDownload = (user?.permissions || []).includes('job_request:download-details');
  const [busy, setBusy] = useState(false);

  async function onDownload() {
    setBusy(true);
    const id = toast.loading('Preparing Job Request PDF…');
    try {
      const { filename } = await downloadJobRequestDetails(jr.id || jr.jr_no);
      toast.success(`Downloaded ${filename}`, { id });
    } catch (e) {
      const msg = e.response?.data?.error?.message
              || e.message
              || 'Failed to download request PDF';
      toast.error(msg, { id });
    } finally {
      setBusy(false);
    }
  }

  return (
    <div className="space-y-4">
      {/* Back-link */}
      <Link
        to="/job-requests"
        className="inline-flex items-center gap-1.5 text-sm text-ink-soft hover:text-ink"
      >
        <ArrowLeft size={14} strokeWidth={1.75} aria-hidden="true" />
        Back to Job Requests
      </Link>

      <div className="flex items-start justify-between gap-4">
        {/* Left: code + meta */}
        <div className="space-y-1.5">
```

> **Reading Note**
>
> Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from FE\src\pages\jobRequests\components\DetailHeader.jsx around line 88

```
            <span>Category: <span className="font-medium text-ink">{jr.job_category || '—'}</span></span>
            <span aria-hidden="true">·</span>
            <span>Submitted: <span className="font-medium text-ink">{fmt(jr.submitted_at || jr.created_at)}</span></span>
            {jr.updated_at && jr.updated_at !== jr.created_at ? (
              <>
                <span aria-hidden="true">·</span>
                <span>Updated: <span className="font-medium text-ink">{fmt(jr.updated_at)}</span></span>
              </>
            ) : null}
          </div>
        </div>

        {/* Right: status + priority pills + Download button (Phase 11) */}
        <div className="flex flex-col items-end gap-1.5">
          <StatusPill status={jr.status} />
          <div className="text-xs text-ink-soft">
            Priority: <PriorityLabel priority={jr.priority} />
          </div>
          {canDownload ? (
            <Button
              variant="secondary"
              size="sm"
              disabled={busy}
              title={`Download Job Request PDF (${jr.request_code})`}
              onClick={onDownload}
            >
              <FileDown size={14} strokeWidth={1.75} aria-hidden="true" />
              {busy ? 'Preparing…' : 'Download Request PDF'}
            </Button>
          ) : null}
        </div>
      </div>
    </div>
  );
}
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Code Appendix: BE\db\discovery\smoke_phase11.js

Technical Purpose

This source file participated in Phase 11. It is included as a guided code extract so that the implementation can be revised, audited, or handed over without opening the project folder immediately.

Code Metric	Value
Lines captured in transcript	284
Layer	Database / Backend / Template / Frontend / QA
Documenting style	Selected chunks with commentary

Extract from BE\db\discovery\smoke_phase11.js around line 1

```
// ========================================================================
// db/discovery/smoke_phase11.js
// ------------------------------------------------------------------------
// PHASE 11 — PDF Generation end-to-end smoke matrix.
//
// CERTIFICATE (PDF #1):
//   C1  VERIFIED_CLOSED JC → 200, application/pdf, single page, filename ok
//   C2  COMPLETED       JC → 200 (if any exist in DB)
//   C3  ASSIGNED        JC → 409 (ineligible), JSON error envelope
//   C4  bogus id        → 404
//   C5  RBAC: VIEW_ONLY → 403 (no job_card:download-certificate)
//   C6  no auth         → 401
//   C7  field mapping   → cross-check (we use bytes+page count proxy,
//                         since PDFKit embeds text as glyph IDs)
//
// JC DETAILS (PDF #2):
//   D1  any JC          → 200, multi-page allowed, filename ok
//   D2  bogus id        → 404
//   D3  RBAC: NORMAL    → 403 (no job_card:download-details)
//   D4  documents list  → only metadata fields appear (no Blob bytes leaked)
//
// JR DETAILS (PDF #3):
//   R1  any JR          → 200, application/pdf
//   R2  bogus id        → 404
//   R3  RBAC row-scope  → NORMAL trying foreign JR → 404 (NOT 403, to avoid
//                         leaking existence — mirrors Phase-7 Slice-2 policy)
//   R4  no auth         → 401
//
// ZERO-WRITE PROOF:
//   Z1  audit_log rows BEFORE = AFTER all PDF requests
//   Z2  cmms_jobcard_mst.updated_at BEFORE = AFTER (verified per JC under test)
//
// REPO ALIASING:
//   A1  grep service/controller/templates for JM_ / JR_ / EQM_ legacy names
//       outside the repo.js — must be zero hits.
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\db\discovery\smoke_phase11.js around line 125

```
  const [as]  = await conn.query("SELECT JM_SectionJobNo FROM cmms_jobcard_mst WHERE JM_MVP_STATUS='ASSIGNED' LIMIT 1");
  const [jrs] = await conn.query("SELECT JR_JOBREQUESTNO, JR_SUBMITTEDBYID FROM cmms_jobrequest_mst ORDER BY JR_JOBREQUESTNO
DESC LIMIT 1");
  const VC_ID = vc[0]?.JM_SectionJobNo;
  const AS_ID = as[0]?.JM_SectionJobNo;
  const JR_ID = jrs[0]?.JR_JOBREQUESTNO;
  const JR_OWNER = jrs[0]?.JR_SUBMITTEDBYID;
  console.log(`${C.gray}Picked: VC_JC=${VC_ID}, AS_JC=${AS_ID}, JR=${JR_ID} (owner=${JR_OWNER})${C.reset}\n`);

  // ── X. No-auth → 401 ──────────────────────────────────────────
  const anon = makeClient();
  for (const [name, url] of [
    ['no-auth cert',        `/job-cards/${VC_ID}/certificate.pdf`],
    ['no-auth JC details',  `/job-cards/${VC_ID}/details.pdf`],
    ['no-auth JR details',  `/job-requests/${JR_ID}/details.pdf`],
  ]) {
    const r = await anon.get(url);
    if (r.status === 401) ok(`${name} → 401`);
    else                  bad(`${name} expected 401`, `got ${r.status}`);
  }

  // ── Snapshot audit_log + JC.updated_at for zero-write proof ────
  const [[{ n: auditBefore }]] = await conn.query("SELECT COUNT(*) AS n FROM audit_log");
  const [[jcBefore]] = await conn.query(
    "SELECT JM_UPDATED_ON FROM cmms_jobcard_mst WHERE JM_SectionJobNo=?",
    [VC_ID],
  );

  // ── C1: Certificate on VERIFIED_CLOSED card ────────────────────
  const c1 = await sa.client.get(`/job-cards/${VC_ID}/certificate.pdf`);
  if (c1.status === 200 && c1.ct.includes('application/pdf') && isPdf(c1.buf)) {
    const pages = pdfPageCount(c1.buf);
    if (pages === 1) {
      ok(`C1 Certificate (VC) → 200, single page`,
         `${c1.bytes} bytes · ${c1.cd}`);
    } else {
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

Extract from BE\db\discovery\smoke_phase11.js around line 250

```
  const leak = [];
  for (const f of filesToScan) {
    const txt = fs.readFileSync(path.join(base, f), 'utf8');
    // Strip block comments and line comments before scanning so comment
    // text like "JM_VERIFIED_ON" doesn't false-positive.
    const code = txt.replace(/\/\*[\s\S]*?\*\//g, '').replace(/\/\/[^\n]*/g, '');
    // Match SQL-style legacy column patterns appearing in CODE.
    const matches = code.match(/\b(JM_[A-Z][A-Za-z_]*|JR_[A-Z][A-Za-z_]*|EQM_[A-Z][A-Za-z_]*|EMM_[A-Z][A-Za-z_]*|SM_[A-Z][A-Za-
z_]*)\b/g) || [];
    // Filter out tokens that legitimately appear in template payload field
    // names — these are CANONICAL JSON keys returned from the repo, e.g.
    // `payload.jr_no` or `JM_SectionJobNo` as a string PARAMETER (not a
    // column ref) inside a string. We allow tokens only inside string
    // literals (single/double/backtick).
    if (matches.length > 0) {
      leak.push({ file: f, tokens: [...new Set(matches)].slice(0, 8) });
    }
  }
  if (leak.length === 0) {
    ok(`A1 repo aliasing — no legacy JM_/JR_/EQM_ tokens outside repo`);
  } else {
    bad(`A1 legacy column tokens leaked into service/controller/templates`,
        JSON.stringify(leak));
  }

  await conn.end();

  // ── Summary ──────────────────────────────────────────────
  console.log(`\n${C.bold}${pass} passed${C.reset}, ${fail > 0 ? `${C.red}${fail} failed${C.reset}` : '0 failed'}\n`);
  process.exit(fail === 0 ? 0 : 1);
}

main().catch((e) => {
  console.error(`${C.red}FATAL${C.reset}`, e);
  process.exit(2);
});
```

Reading Note

Focus on the boundary: what this file knows, what it refuses to know, and how it supports the Phase 11 sealed contract.

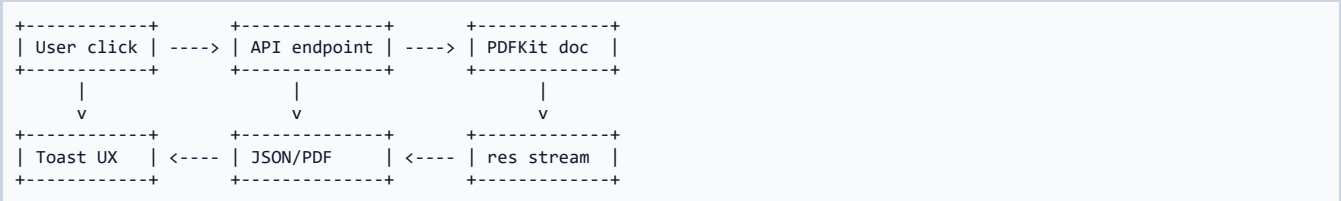
Scenario 1 - Super Admin downloads certificate

End-to-End Story

Super Admin opens a VERIFIED_CLOSED Job Card and clicks Download Report. Frontend requests certificate.pdf. Backend authenticates, authorizes, validates, loads full Job Card, confirms eligible status, sets PDF headers, and streams a single-page PDF.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



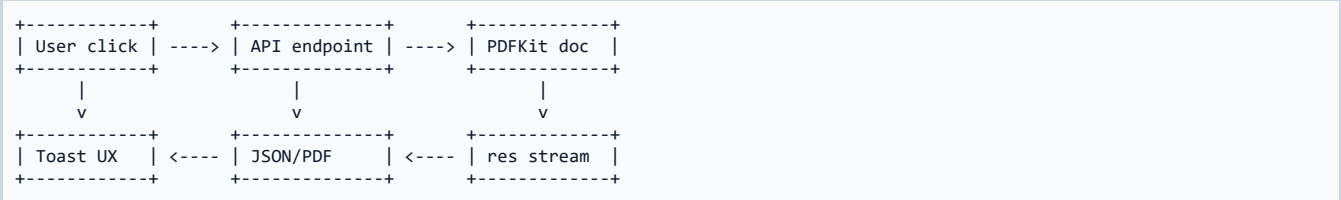
Scenario 2 - Lab Engineer downloads full details

End-to-End Story

Lab Engineer opens an assigned or completed Job Card and downloads details.pdf. Because details are internal working documents, multi-page output is allowed and the renderer includes all tabs and history.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



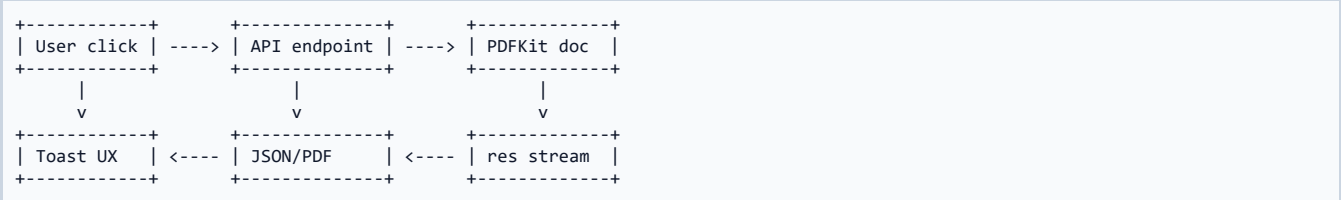
Scenario 3 - View Only user attempts certificate

End-to-End Story

View Only user can inspect details but cannot download the formal certificate. Backend returns 403 even if the UI is bypassed.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



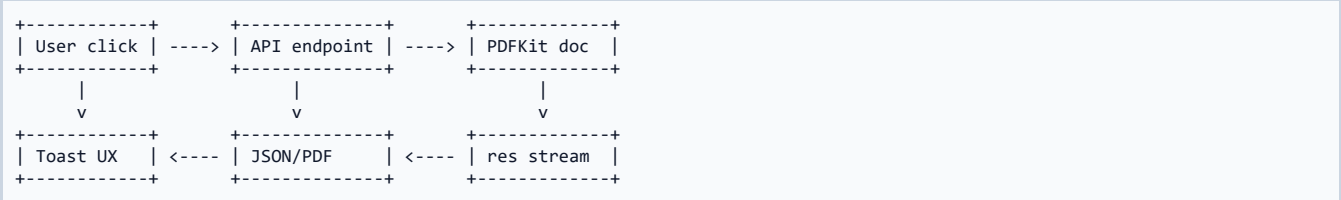
Scenario 4 - Normal User downloads own Job Request

End-to-End Story

Normal User with job_request:download-details can download a PDF for their own request. The backend allows it only after row-scope check passes.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



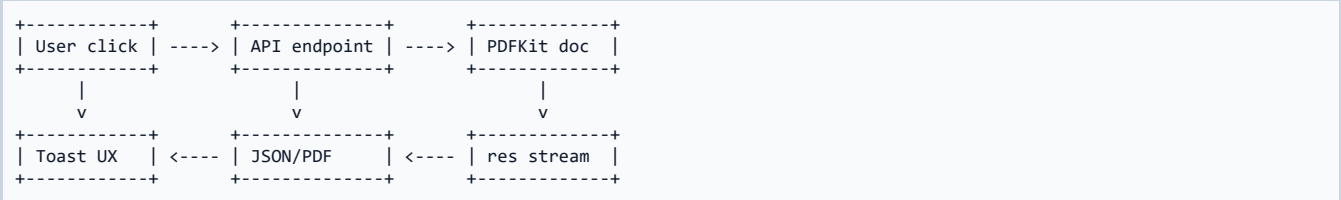
Scenario 5 - Normal User tries foreign Job Request

End-to-End Story

The backend returns 404 instead of 403. This is an intentional no-leak pattern: the system does not confirm whether the foreign request exists.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



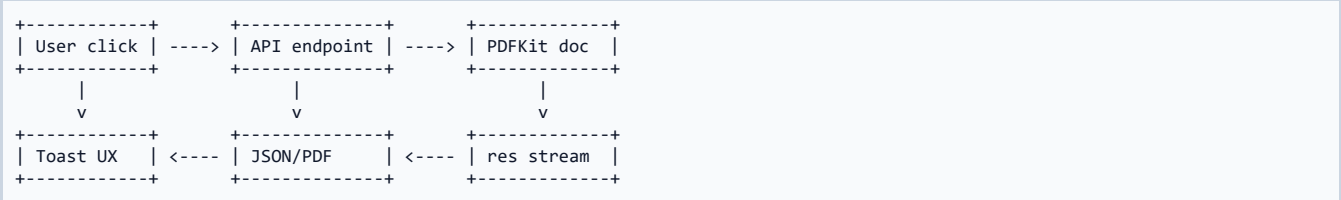
Scenario 6 - Ineligible Job Card certificate

End-to-End Story

ASSIGNED or IN_PROGRESS Job Cards cannot produce the certificate. The API returns 409 CONFLICT with current_status and eligible values.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



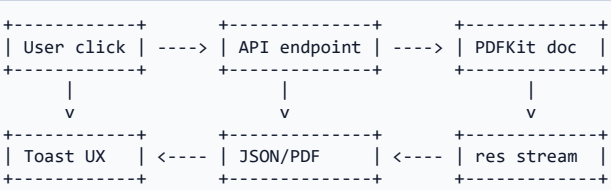
Scenario 7 - Missing Job Card

End-to-End Story

Bogus Job Card section IDs return 404. No PDF stream starts before the not-found check.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



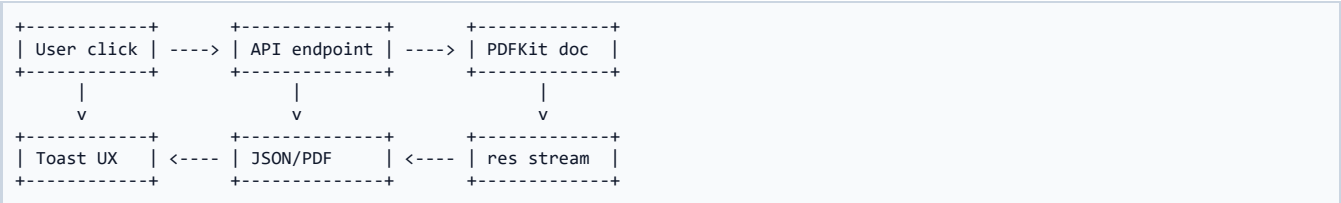
Scenario 8 - Missing Job Request

End-to-End Story

Bogus Job Request IDs return 404. The route validates numeric ID before repository access.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



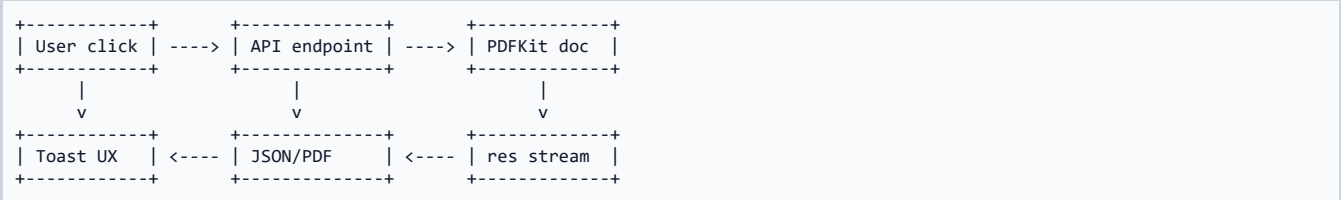
Scenario 9 - PDF error JSON Blob

End-to-End Story

The frontend requested responseType blob; if the backend returns JSON error, the helper reads the Blob text, parses JSON, and shows a readable toast.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



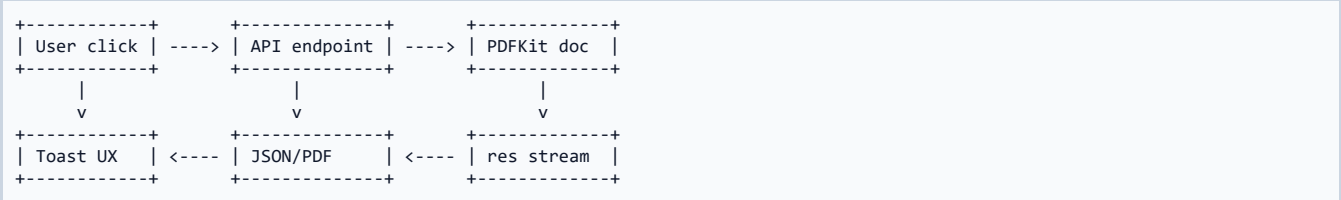
Scenario 10 - Future logo assets

End-to-End Story

If official logo PNG/JPG files are added to assets, the shared header picks them up automatically. If absent, typographic seals maintain layout stability.

Actor	Route	Main Gate	Expected Result
User role	PDF endpoint	Auth + permission + business rule	PDF or structured JSON error
Frontend	api/pdf.js	Blob download wrapper	Browser file save + toast
Backend	pdf.service prepare	Existence + eligibility + row scope	Safe stream start
Database	repo read model	SELECT only	No write side effect

Scenario Diagram



Operational Handover - What is Safe to Change

Handover Note

Template copy, spacing, typography, and non-business wording may be refined if the locked certificate layout remains approved. Details PDFs can receive new sections if new child tables are sealed later.

Checklist Item	Pass Rule
Migrations	410 has seeded the three permissions and role grants
Routes	No-auth returns 401; authorized route returns application/pdf
Certificate	Only COMPLETED / VERIFIED_CLOSED returns 200
Details	JC and JR details download correctly
Zero writes	audit_log and JC updated timestamp remain unchanged
FE build	Vite build completes without blocking errors

Operational Handover - What Must Not Change

Handover Note

Do not persist PDFs, do not log downloads unless a future audit requirement explicitly reopens this decision, do not emit certificate for ineligible states, and do not move legacy SQL column names outside repository files.

Checklist Item	Pass Rule
Migrations	410 has seeded the three permissions and role grants
Routes	No-auth returns 401; authorized route returns application/pdf
Certificate	Only COMPLETED / VERIFIED_CLOSED returns 200
Details	JC and JR details download correctly
Zero writes	audit_log and JC updated timestamp remain unchanged
FE build	Vite build completes without blocking errors

Operational Handover - How to Debug PDF Failures

Handover Note

Start with route status, then auth/permissions, then validator result, then service prepare result, then repository payload, then renderer page count. Never debug by bypassing authorization.

Checklist Item	Pass Rule
Migrations	410 has seeded the three permissions and role grants
Routes	No-auth returns 401; authorized route returns application/pdf
Certificate	Only COMPLETED / VERIFIED_CLOSED returns 200
Details	JC and JR details download correctly
Zero writes	audit_log and JC updated timestamp remain unchanged
FE build	Vite build completes without blocking errors

Operational Handover - How to Extend

Handover Note

Add a permission, add a validator if needed, add a repo read model, add a template, add a prepare service, wire controller and route, add frontend wrapper, then extend smoke matrix.

Checklist Item	Pass Rule
Migrations	410 has seeded the three permissions and role grants
Routes	No-auth returns 401; authorized route returns application/pdf
Certificate	Only COMPLETED / VERIFIED_CLOSED returns 200
Details	JC and JR details download correctly
Zero writes	audit_log and JC updated timestamp remain unchanged
FE build	Vite build completes without blocking errors

Operational Handover - Deployment Reminder

Handover Note

Run migrations, restart backend, login with representative roles, test all three endpoints, verify download filenames, verify ineligible certificate 409, and verify frontend toasts.

Checklist Item	Pass Rule
Migrations	410 has seeded the three permissions and role grants
Routes	No-auth returns 401; authorized route returns application/pdf
Certificate	Only COMPLETED / VERIFIED_CLOSED returns 200
Details	JC and JR details download correctly
Zero writes	audit_log and JC updated timestamp remain unchanged
FE build	Vite build completes without blocking errors

Final Phase 11 Seal

SEALED RESULT

Phase 11 is sealed as the CMCMIS PDF Generation phase. It transforms live sealed operational records into three controlled documents while preserving read-only behavior, permission safety, status eligibility, row-level privacy, and frontend usability.

Final Question	Answer
Were three PDFs completed?	Yes
Was the Job Card Certificate locked to a single page?	Yes
Were details PDFs completed?	Yes - JC Details and JR Details
Was the Download Report button enabled?	Yes
Was smoke verification green?	Yes - 23/23
Was zero-write behavior proven?	Yes
Is Phase 11 ready for documentation seal?	Yes

DS Closing Note

This document should be treated as the Phase 11 reference. Future phases must not reinterpret Phase 11 as a reports module or as a workflow transition module. It is the official PDF generation layer over sealed job request and job card data.